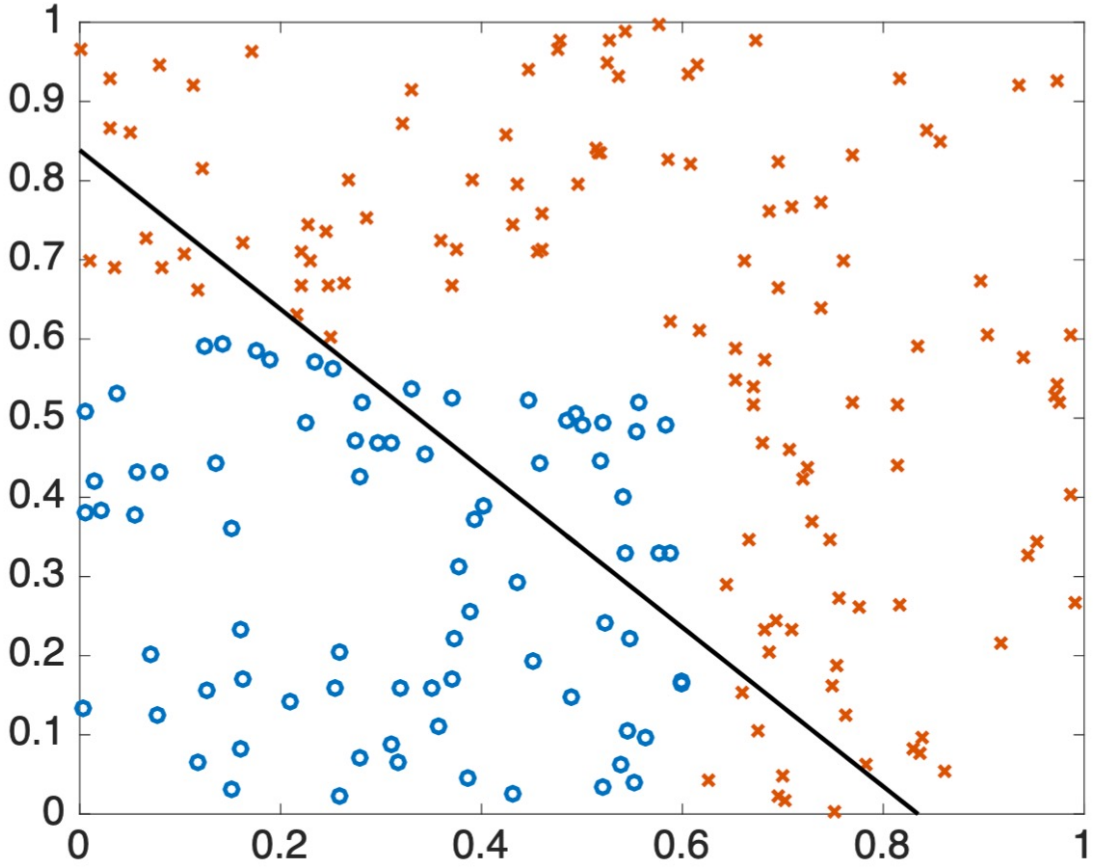
AdaBoost

Lecture Note on Boosting

AARHUS
UNIVERSITY

Issues with Linear Models

Best logistic regression classifier

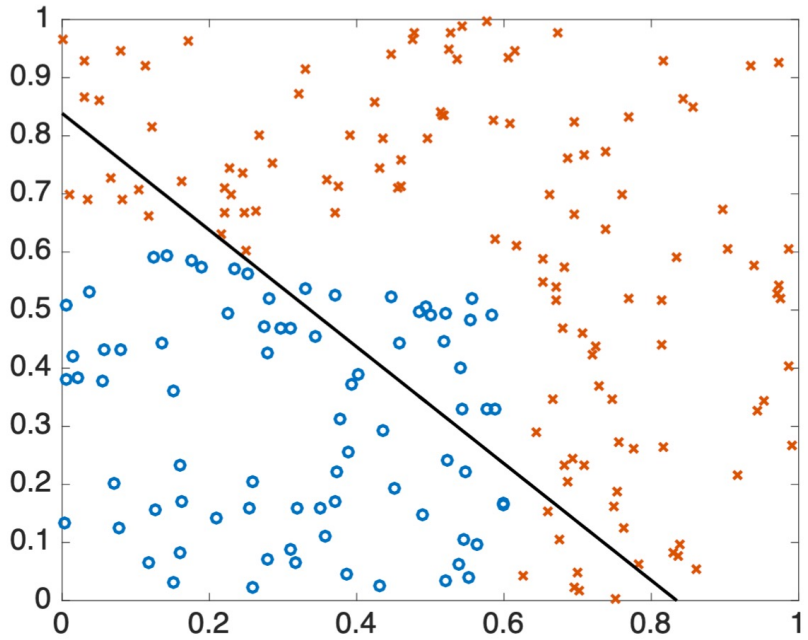


Feature transform could work:

- (x,y)->(max(x,y))

Issues with Linear Models

- Feature transform could work:
  - (x,y)->(max(x,y))



- Issues:
  - Risk overfitting.
  - Need to hand-craft features.
  - Used knowledge of data set.
  - Also: Gradient descent sensitive to scaling.

- Without feature transform:
  - Only a bit better than random guessing.
  - A "weak" learner.



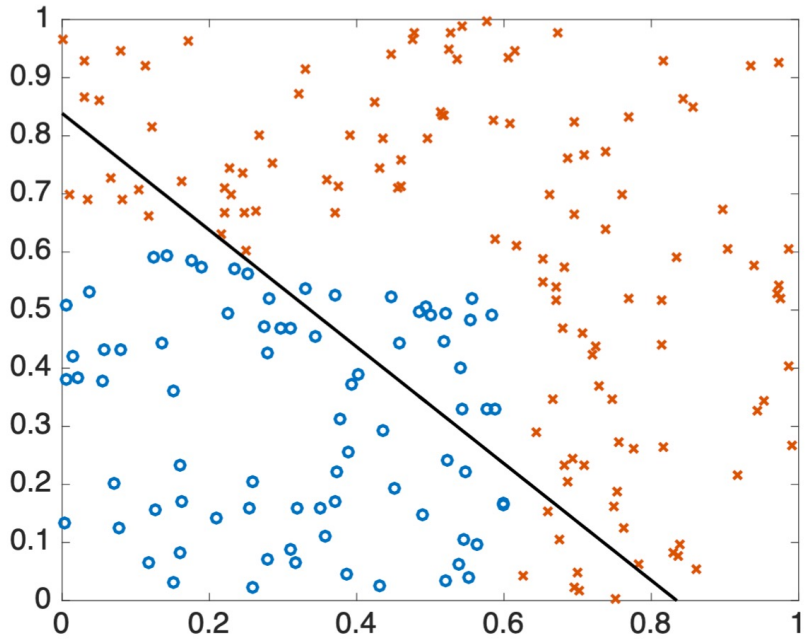
Boosting

Make Weak Learners into Strong Learners

Focus on Binary Classification

Boosting

- Weak learner is a classifier that does slightly better than guessing on training set.
- Strong learner is a classifier that perfectly classifies training set.
- Boosting:
  - Make weak learner into strong learner.



- How?
  - Combine weak learners into committee/ensemble.
  - Vote like in bagging and random forest.

Boosting

- Meta Algorithm
  - Boosting takes any learning algorithm and builds a new one on top of it.
    - Base learning algorithm A.
    - Base hypothesis set H.

- Boosting:
  - Run A T times (not on same data set), producing hypotheses $h_1,\ldots,h_T \in H$.
  - Compute weights $\alpha_1,\ldots,\alpha_T$.
  - Return voting classifier:
    - $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$.



Boosting

- Boosting:
  - Run A T times (not on same data set), producing hypotheses $h_1,\ldots,h_T \in H$.
  - Compute weights $\alpha_1,\ldots,\alpha_T$.
  - Return voting classifier:
    - $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$.

Difference lies in datasets and weights

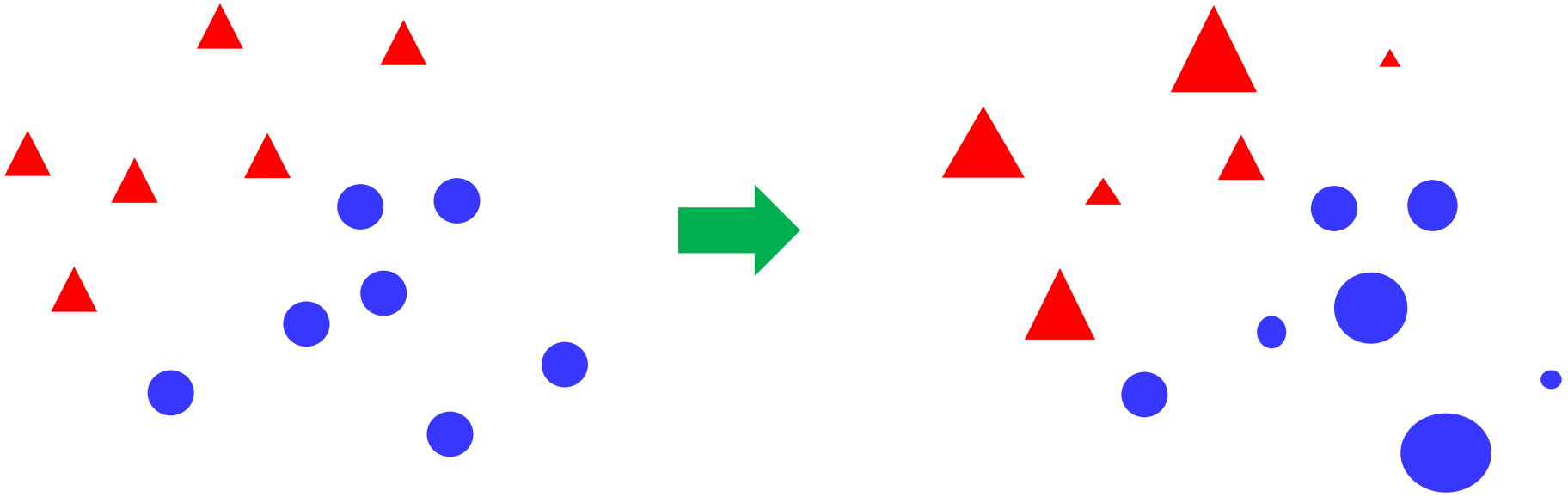
- Bagging/Random Forest:
  - Run A T times (on subsampled datasets), producing hypotheses $h_1,\ldots,h_T \in H$.
  - Return voting classifier:
    - $f(x) = \text{sign}(\sum_{t=1}^{T} h_t(x))$.



Weighted Data Sets

Learning algorithms seen so far easily adapt to weighted setup.

- Weigh points differently:
  - Training set $(x_1,y_1),\ldots,(x_n,y_n)$. $X=\{x_1,\ldots,x_n\}$.
  - Probability distribution D over X, i.e. $\sum_{i=1}^{n} D(i) = 1$.
  - Minimize weighted 0-1 loss $L_D(h) = \sum_{i=1}^{n} D(i) 1_{h(x_i) \neq y_i}$.
    - D uniform gives normal 0-1 loss.



Boosting

- Boosting:
  - Run A T times on reweighed datasets, producing hypotheses $h_1,\ldots,h_T \in H$.
  - Compute weights $\alpha_1,\ldots,\alpha_T$.
  - Return voting classifier:
    - $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$.



Boosting

- Bagging/Random Forest:
  - Run A T times (on subsampled datasets), producing hypotheses $h_1,\ldots,h_T \in H$.
  - Return voting classifier:
    - $f(x) = \text{sign}(\sum_{t=1}^{T} h_t(x))$.

- Same as reweighing:
  - Recall bagging samples n' training examples $(x_1',y_1'),\ldots,(x_{n'}',y_{n'}')$ with replacement.
  - If original example (x_i,y_i) is sampled t_i times, define $D(i)=t_i/n'$.
  - 0-1 loss on subsample same as weighted 0-1 loss.



Boosting

- For hypothesis h:
  - $\frac{1}{n'}\sum_{i=1}^{n'} 1_{h(x_i')\neq y_i'} =$
  - $\frac{1}{n'}\sum_{i=1}^{n} t_i 1_{h(x_i)\neq y_i} =$
  - $\sum_{i=1}^{n} \frac{t_i}{n'} 1_{h(x_i)\neq y_i} =$
  - $\sum_{i=1}^{n} D(i) 1_{h(x_i)\neq y_i} = L_D(h)$

- Same as reweighing:
  - Recall bagging samples n' training examples $(x_1',y_1'),\ldots,(x_{n'}',y_{n'}')$ with replacement.
  - If original example (x_i,y_i) is sampled t_i times, define $D(i)=t_i/n'$.
  - 0-1 loss on subsample same as weighted 0-1 loss.



Boosting

- Boosting:
  - Run A T times on reweighed datasets, producing hypotheses $h_1,\ldots,h_T \in H$.
  - Compute weights $\alpha_1,\ldots,\alpha_T$.
  - Return voting classifier:
    - $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$.

- Questions:
  - How to choose the weighing on training data in i'th iteration.
  - How to choose the weights $\alpha_1,\ldots,\alpha_T$ on hypotheses.



AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.
- Output: $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.

- Output: $f(x) = \mathrm{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$

Start with uniform weighing



AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.

Compute weighted loss of h_t

- Output: $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.

"Magic" constant

- Output: $f(x) = \mathrm{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.

Reweigh for next iteration

- Output: $f(x) = \mathrm{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.

Normalization factor so sum to 1

- Output: $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



AdaBoost

- Discussion: New weights set to
  - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

    $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.

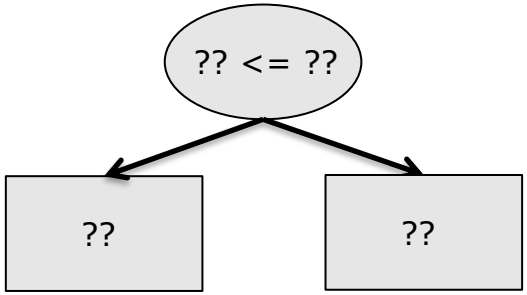
- Observation:
  - If $y_i = h_t(x_i)$ then change by $e^{-\alpha_t}/Z_t$ ↓
  - If $y_i \neq h_t(x_i)$ then change by e^{α_t}/Z_t ↑

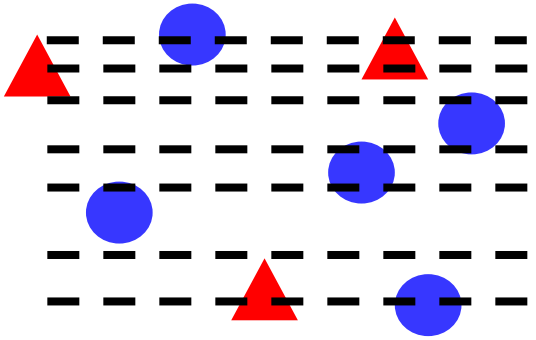
- Intuition:
  - Put more emphasis on points misclassified.
  - Base algorithm A will focus its efforts on points repeatedly misclassified during previous iterations.



AdaBoost with Decision Stumps

- Example: Let us see an example execution.
  - A = train best Decision Stump.
    - One root and two children that are leaves.



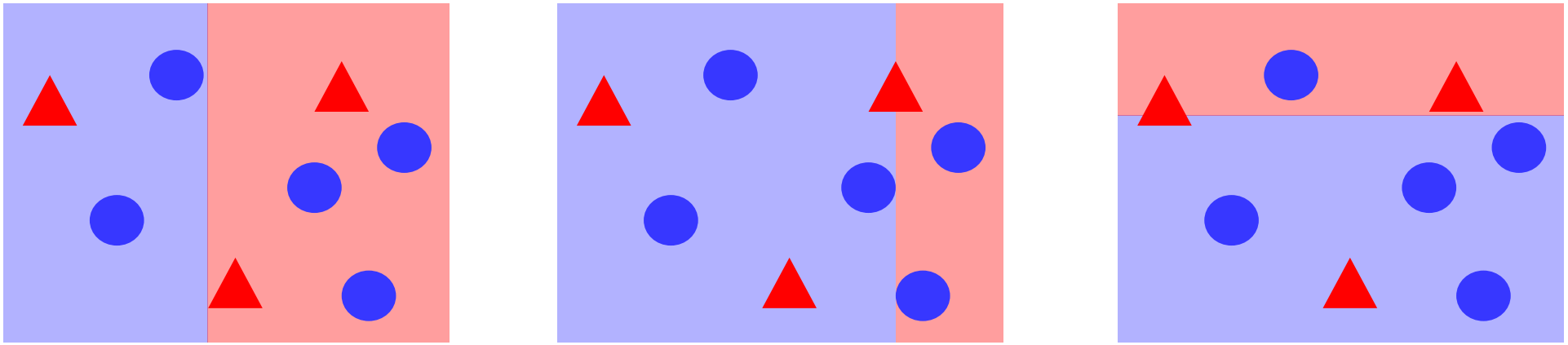


- What is best choice of root?
  - For every feature, there are n-1 relevant splits with n training points.
  - Try all:
    - For each, choose leaves to minimize L_{D_t}.

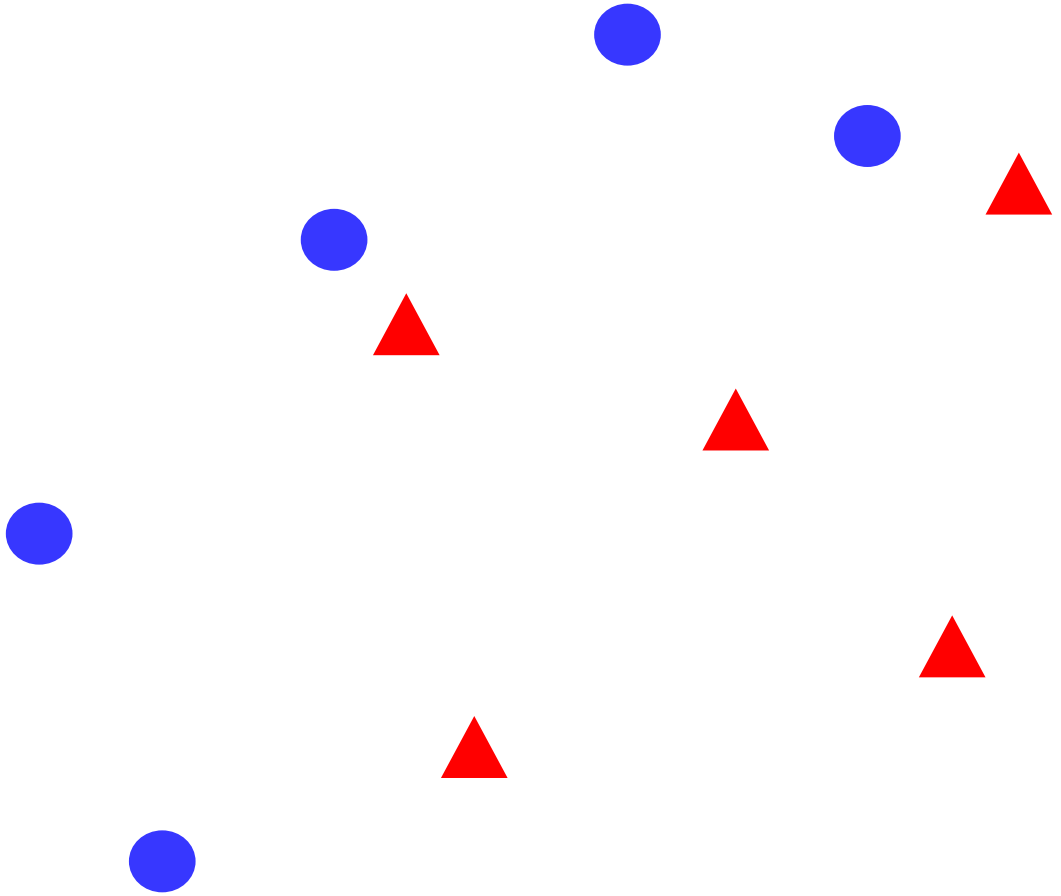


AdaBoost with Decision Stumps

- Decision Stumps
  - Hypothesis set: All axis-parallel hyperplanes/lines



AdaBoost with Decision Stumps



AdaBoost

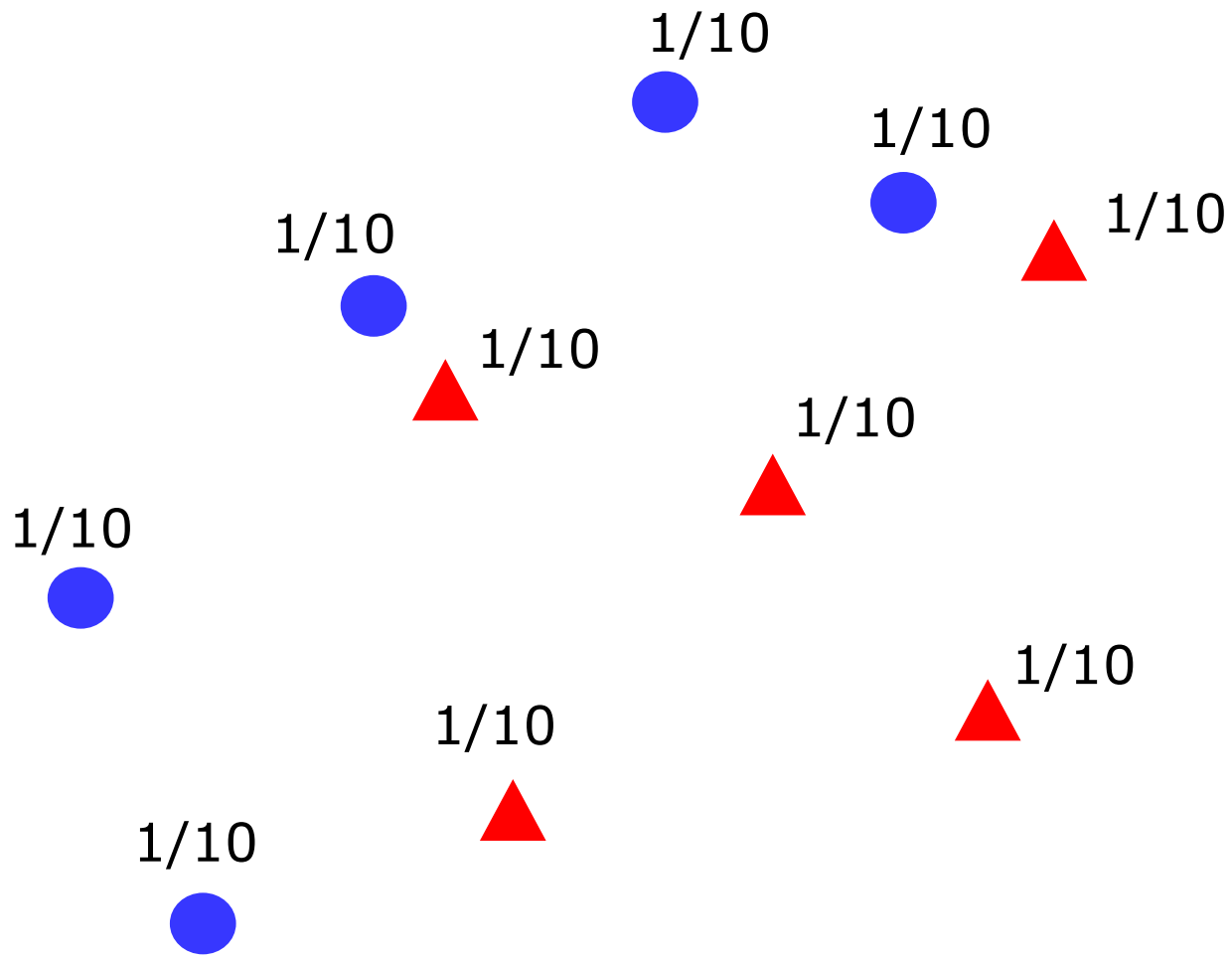
- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.
- Output: $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



AdaBoost with Decision Stumps

D_1 uniform



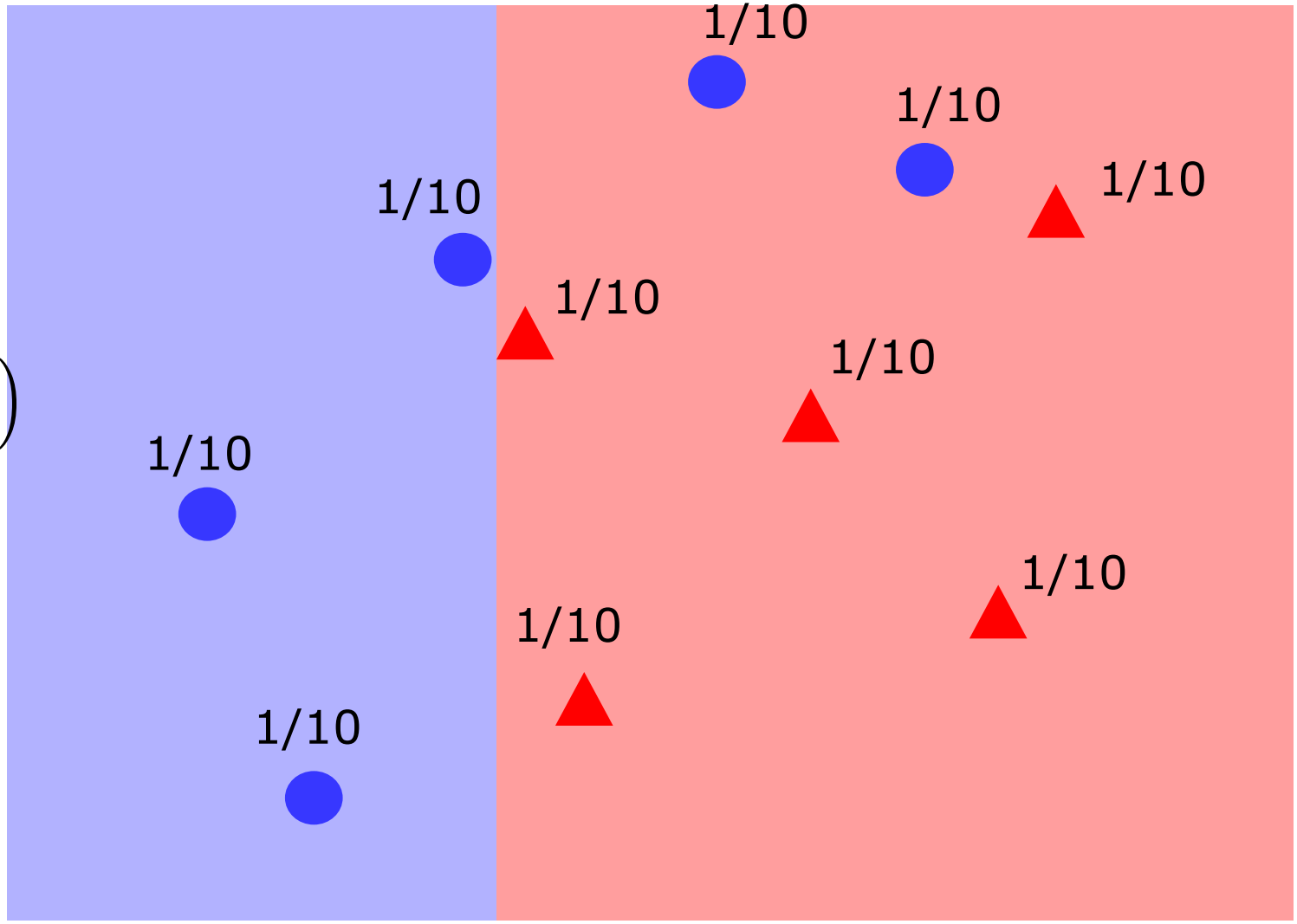
AdaBoost with Decision Stumps

$er_1 = 2/10$

$\alpha_1 = \frac{1}{2}\ln\left(\frac{1-er_1}{er_1}\right) = \frac{1}{2}\ln\left(\frac{8}{2}\right) \approx 0.69$

$e^{\alpha_1} = 2$

$e^{-\alpha_1} = 1/2$



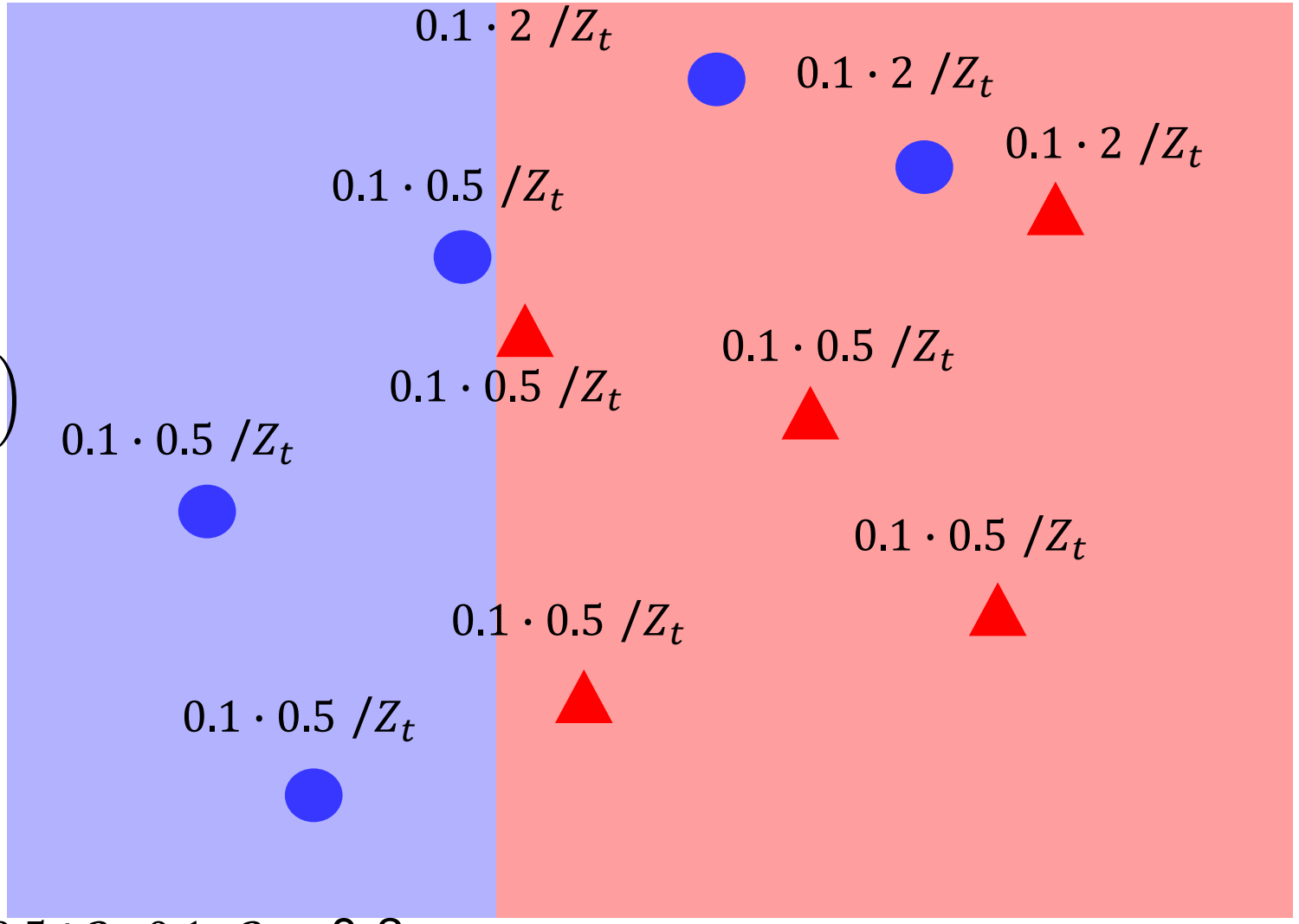
AdaBoost with Decision Stumps

$er_1 = 2/10$

$\alpha_1 = \frac{1}{2}\ln\left(\frac{1-er_1}{er_1}\right) = \frac{1}{2}\ln\left(\frac{8}{2}\right) \approx 0.69$

$e^{\alpha_1} = 2$

$e^{-\alpha_1} = 1/2$



$Z_1 = 8 \cdot 0.1 \cdot 0.5 + 2 \cdot 0.1 \cdot 2 = 0.8$



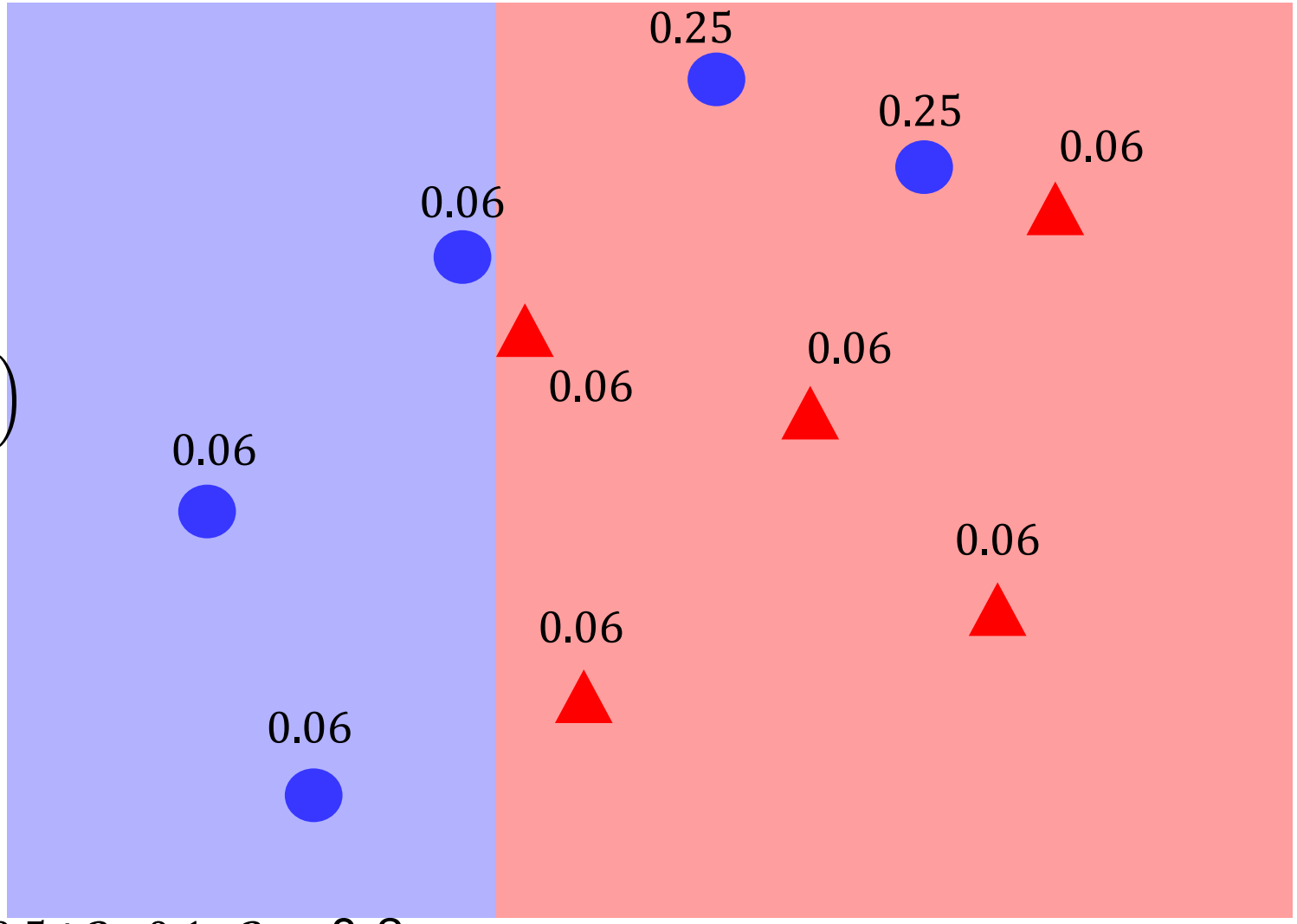
AdaBoost with Decision Stumps

$er_1 = 2/10$

$\alpha_1 = \frac{1}{2}\ln\left(\frac{1-er_1}{er_1}\right) = \frac{1}{2}\ln\left(\frac{8}{2}\right) \approx 0.69$

$e^{\alpha_1} = 2$

$e^{-\alpha_1} = 1/2$



$Z_1 = 8 \cdot 0.1 \cdot 0.5 + 2 \cdot 0.1 \cdot 2 = 0.8$

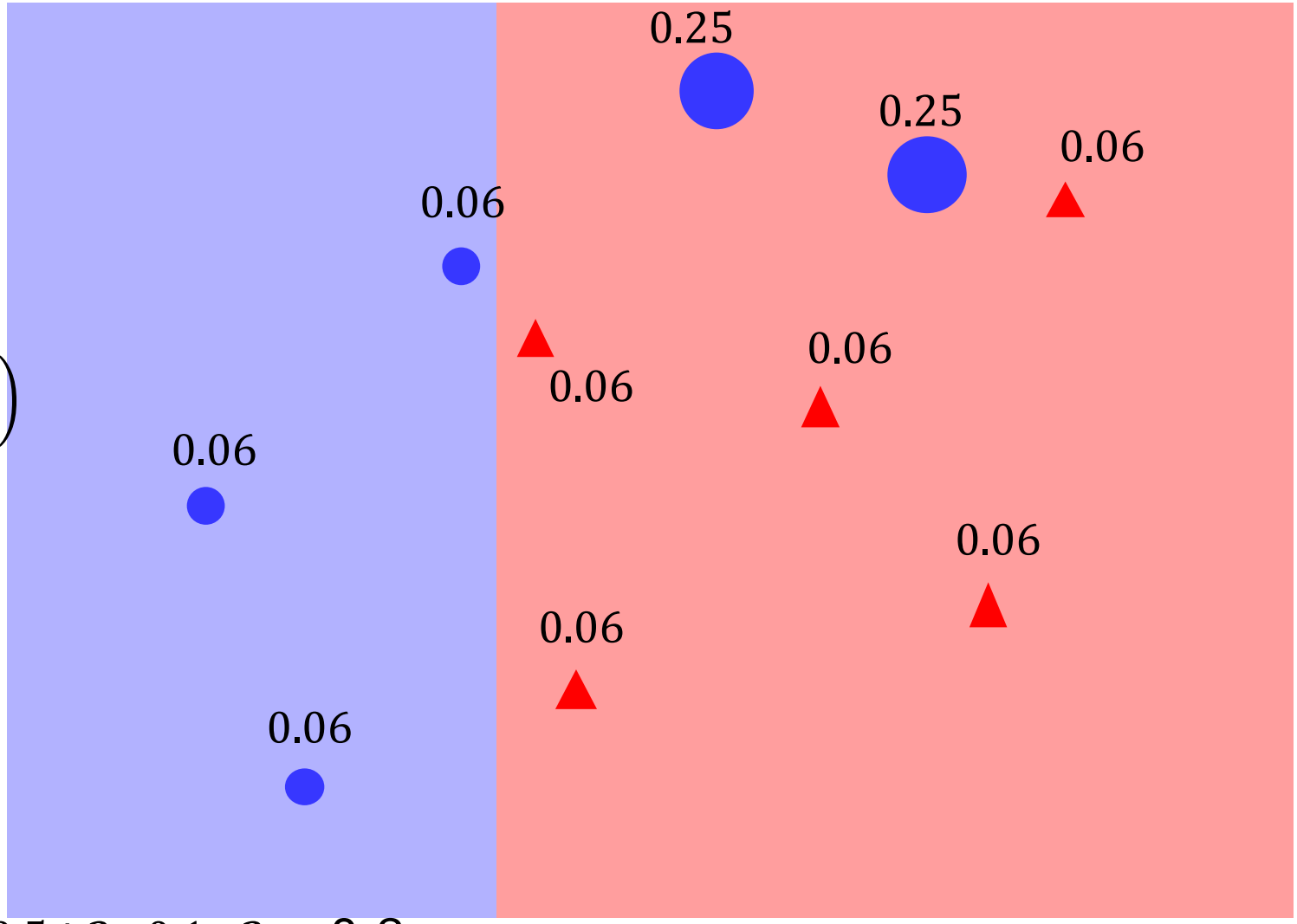
AdaBoost with Decision Stumps

$er_1 = 2/10$

$\alpha_1 = \frac{1}{2}\ln\left(\frac{1-er_1}{er_1}\right) = \frac{1}{2}\ln\left(\frac{8}{2}\right) \approx 0.69$

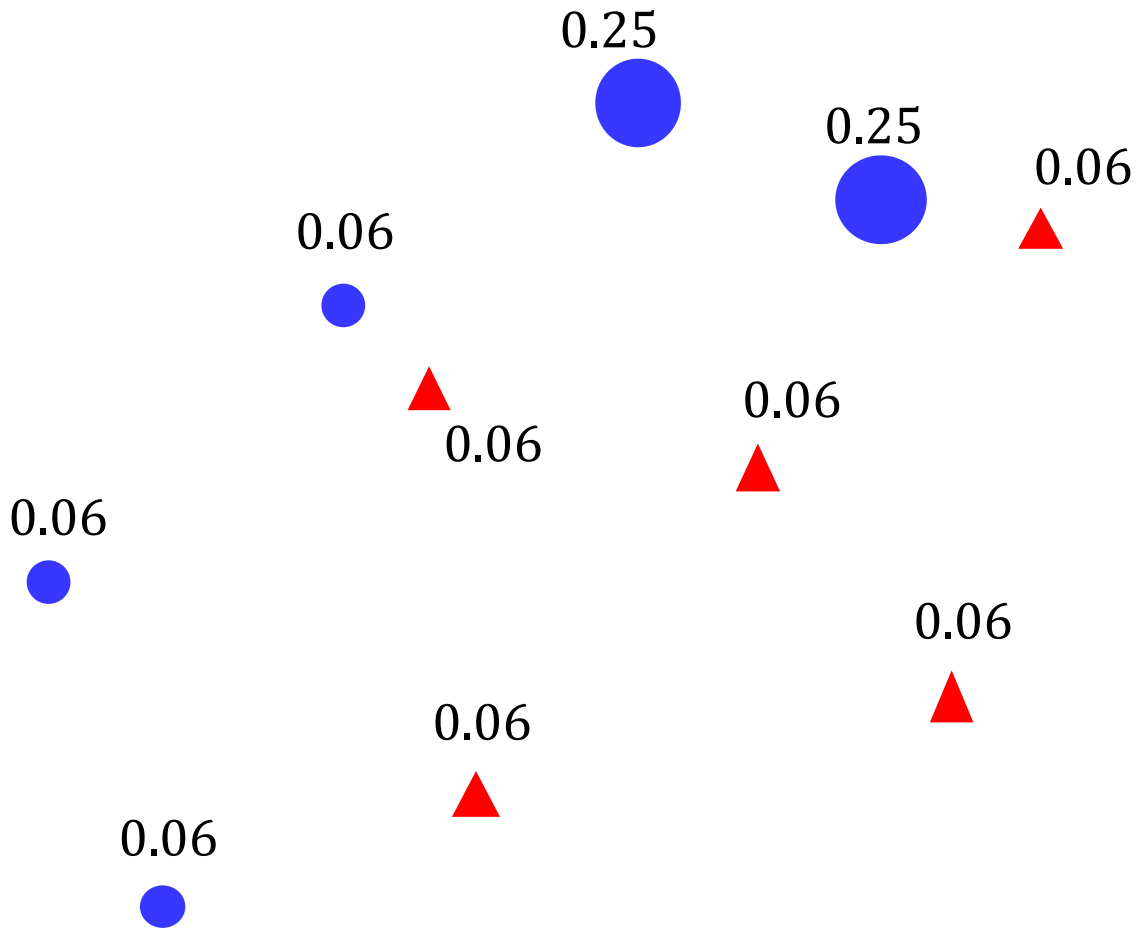
$e^{\alpha_1} = 2$

$e^{-\alpha_1} = 1/2$



$Z_1 = 8 \cdot 0.1 \cdot 0.5 + 2 \cdot 0.1 \cdot 2 = 0.8$

AdaBoost with Decision Stumps



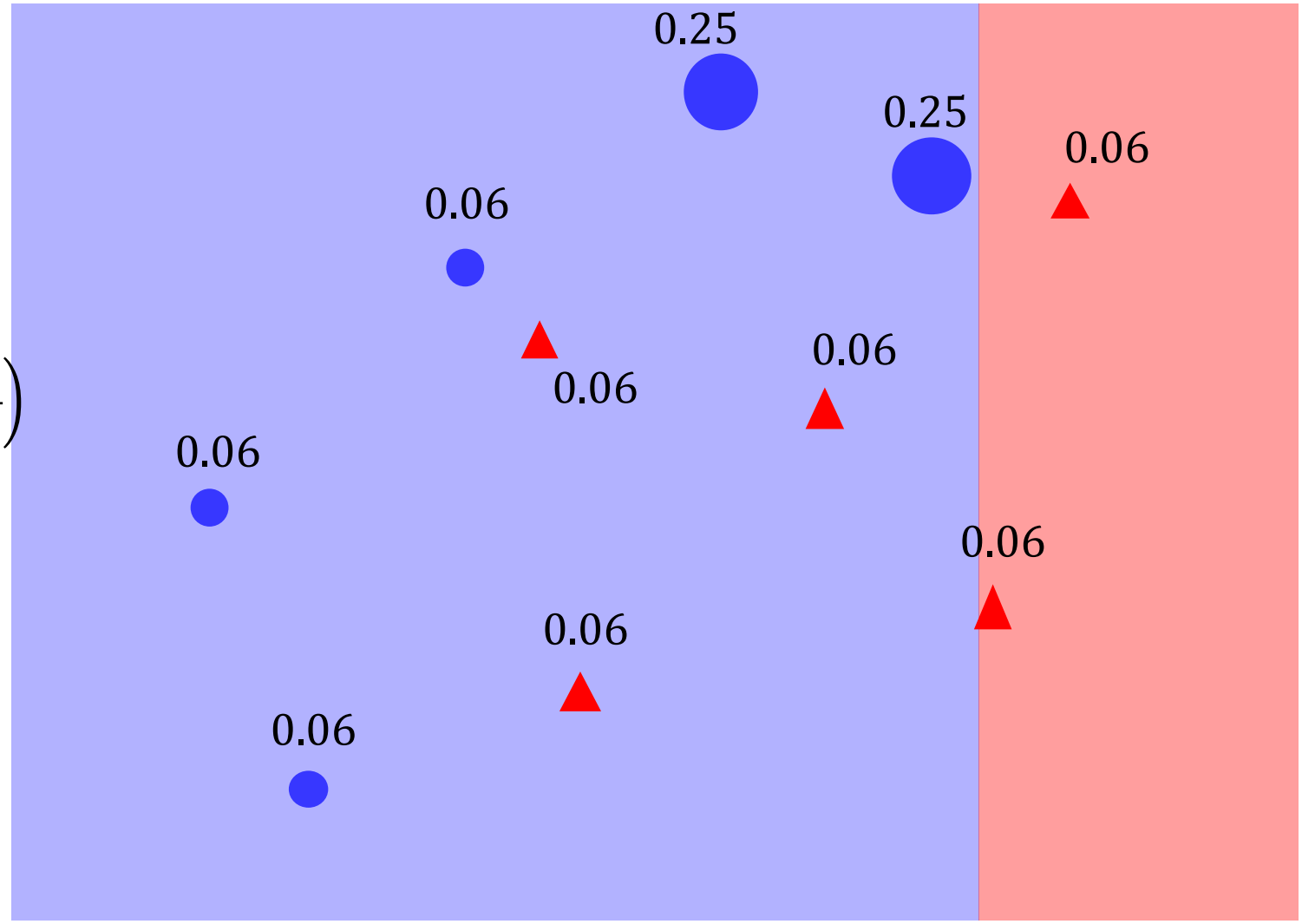
AdaBoost with Decision Stumps

$er_2 \approx 0.19$

$\alpha_2 = \frac{1}{2}\ln\left(\frac{1-er_2}{er_2}\right) \approx 0.73$

$e^{\alpha_2} \approx 2.08$

$e^{-\alpha_2} \approx 0.48$



$Z_2 \approx 2 \cdot 0.25 \cdot 0.48 + 3 \cdot 0.06 \cdot 2.08 + 5 \cdot 0.06 \cdot 0.48 \approx 0.76$

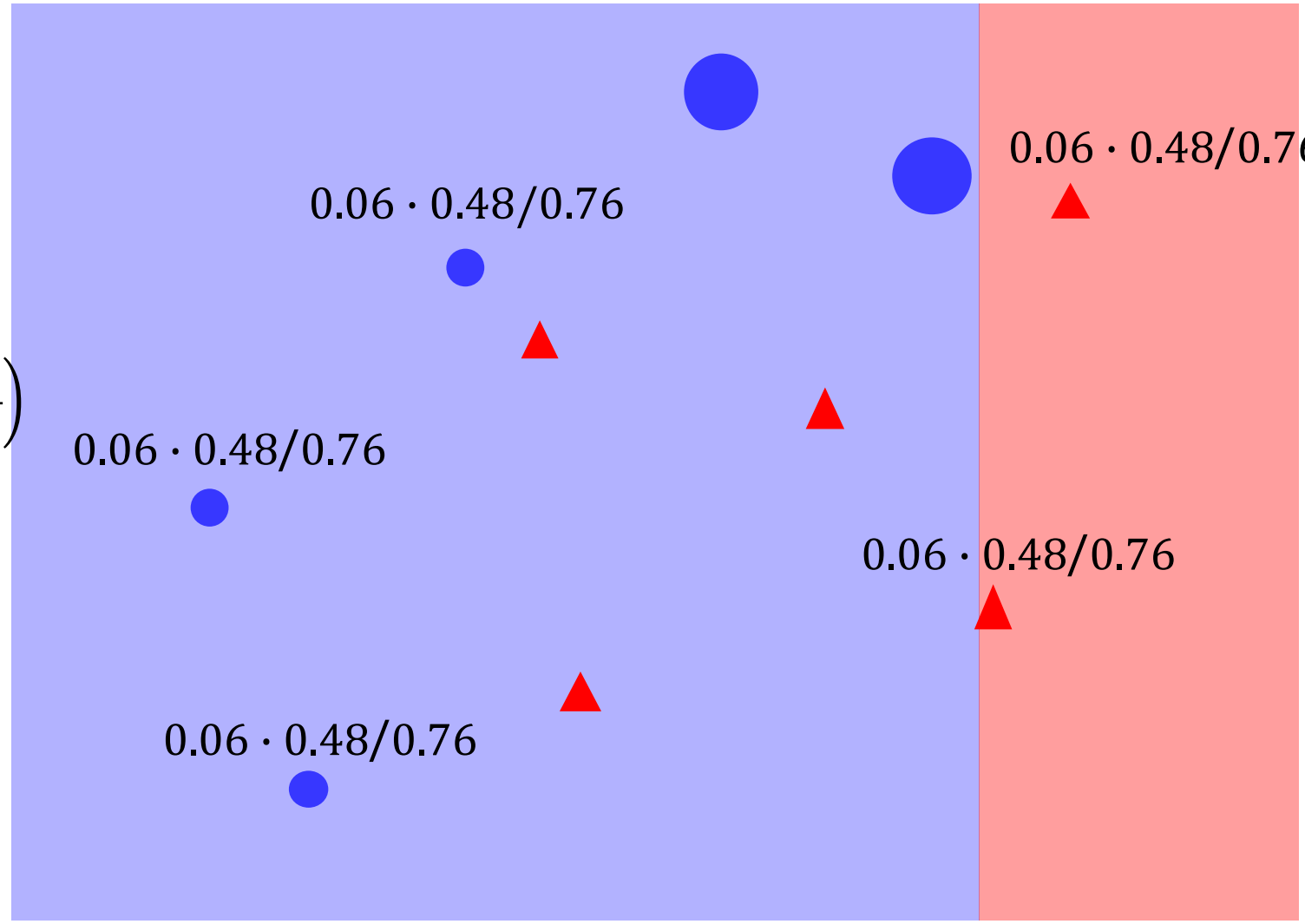
AdaBoost with Decision Stumps

$er_2 \approx 0.19$

$\alpha_2 = \frac{1}{2}\ln\left(\frac{1-er_2}{er_2}\right) \approx 0.73$

$e^{\alpha_2} \approx 2.08$

$e^{-\alpha_2} \approx 0.48$



$Z_2 \approx 2 \cdot 0.25 \cdot 0.48 + 3 \cdot 0.06 \cdot 2.08 + 5 \cdot 0.06 \cdot 0.48 \approx 0.76$

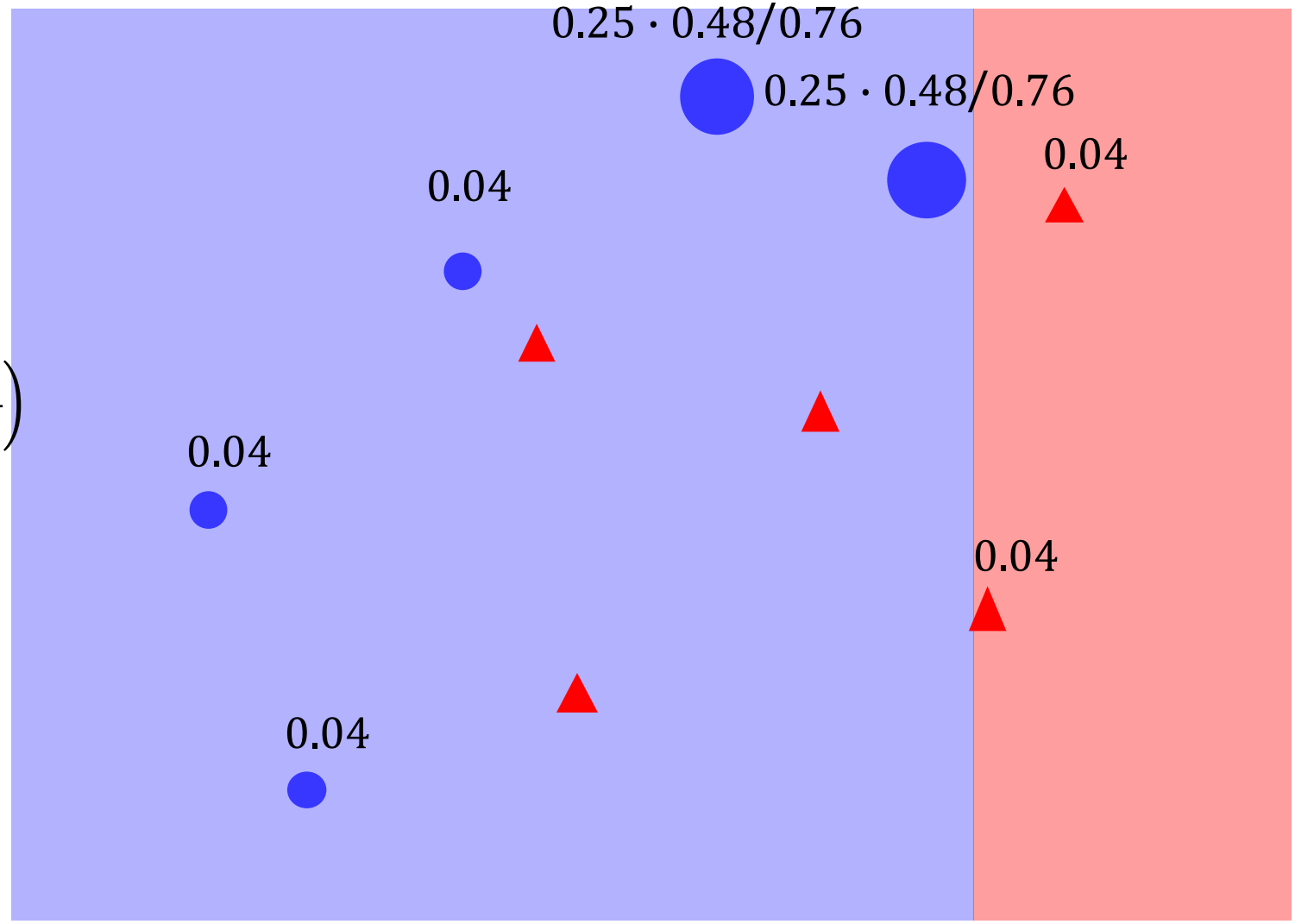
AdaBoost with Decision Stumps

$er_2 \approx 0.19$

$\alpha_2 = \frac{1}{2}\ln\left(\frac{1-er_2}{er_2}\right) \approx 0.73$

$e^{\alpha_2} \approx 2.08$

$e^{-\alpha_2} \approx 0.48$



$Z_2 \approx 2 \cdot 0.25 \cdot 0.48 + 3 \cdot 0.06 \cdot 2.08 + 5 \cdot 0.06 \cdot 0.48 \approx 0.76$

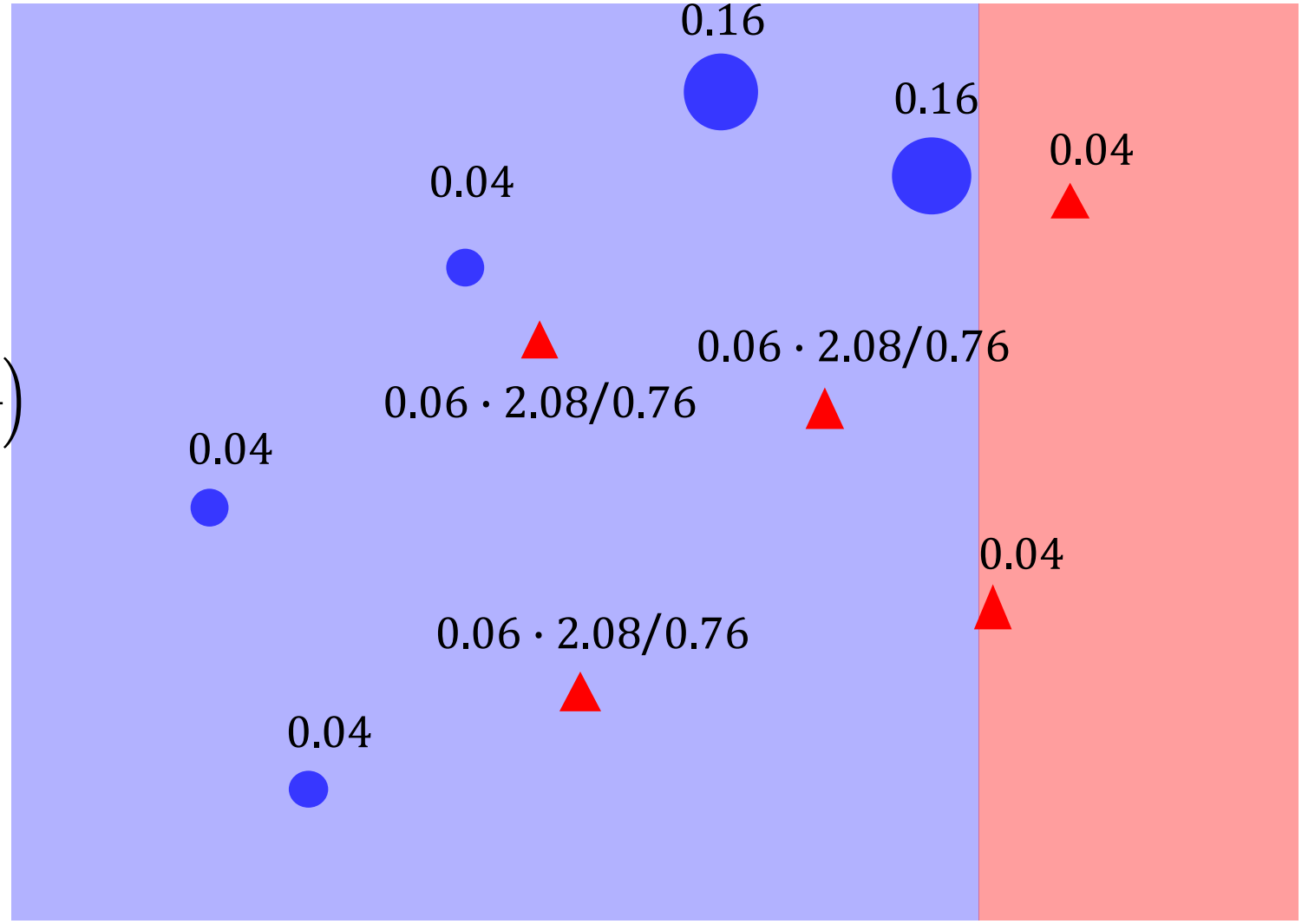
AdaBoost with Decision Stumps

$er_2 \approx 0.19$

$\alpha_2 = \frac{1}{2}\ln\left(\frac{1-er_2}{er_2}\right) \approx 0.73$

$e^{\alpha_2} \approx 2.08$

$e^{-\alpha_2} \approx 0.48$



$Z_2 \approx 2 \cdot 0.25 \cdot 0.48 + 3 \cdot 0.06 \cdot 2.08 + 5 \cdot 0.06 \cdot 0.48 \approx 0.76$

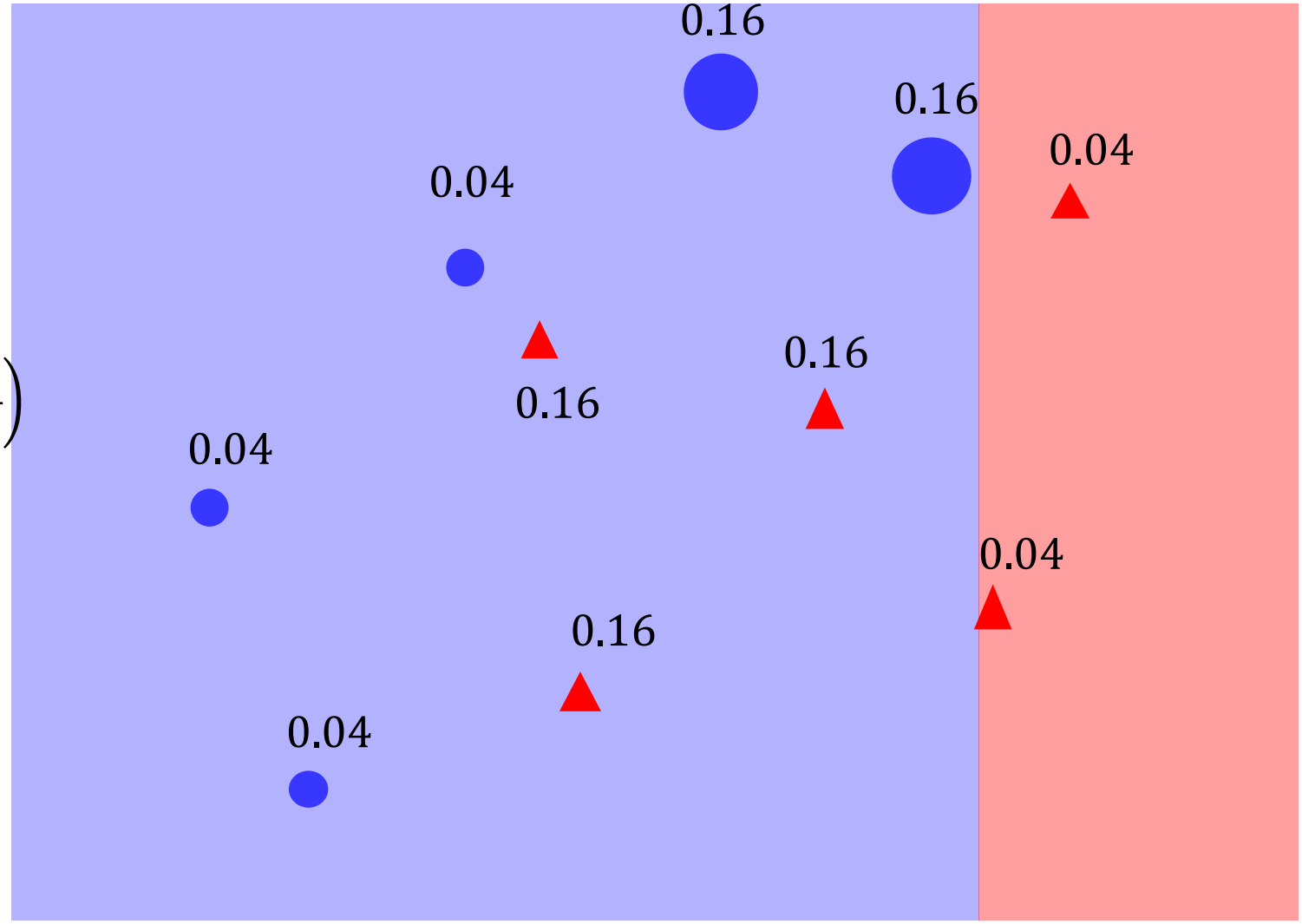
AdaBoost with Decision Stumps

$er_2 \approx 0.19$

$\alpha_2 = \frac{1}{2}\ln\left(\frac{1-er_2}{er_2}\right) \approx 0.73$

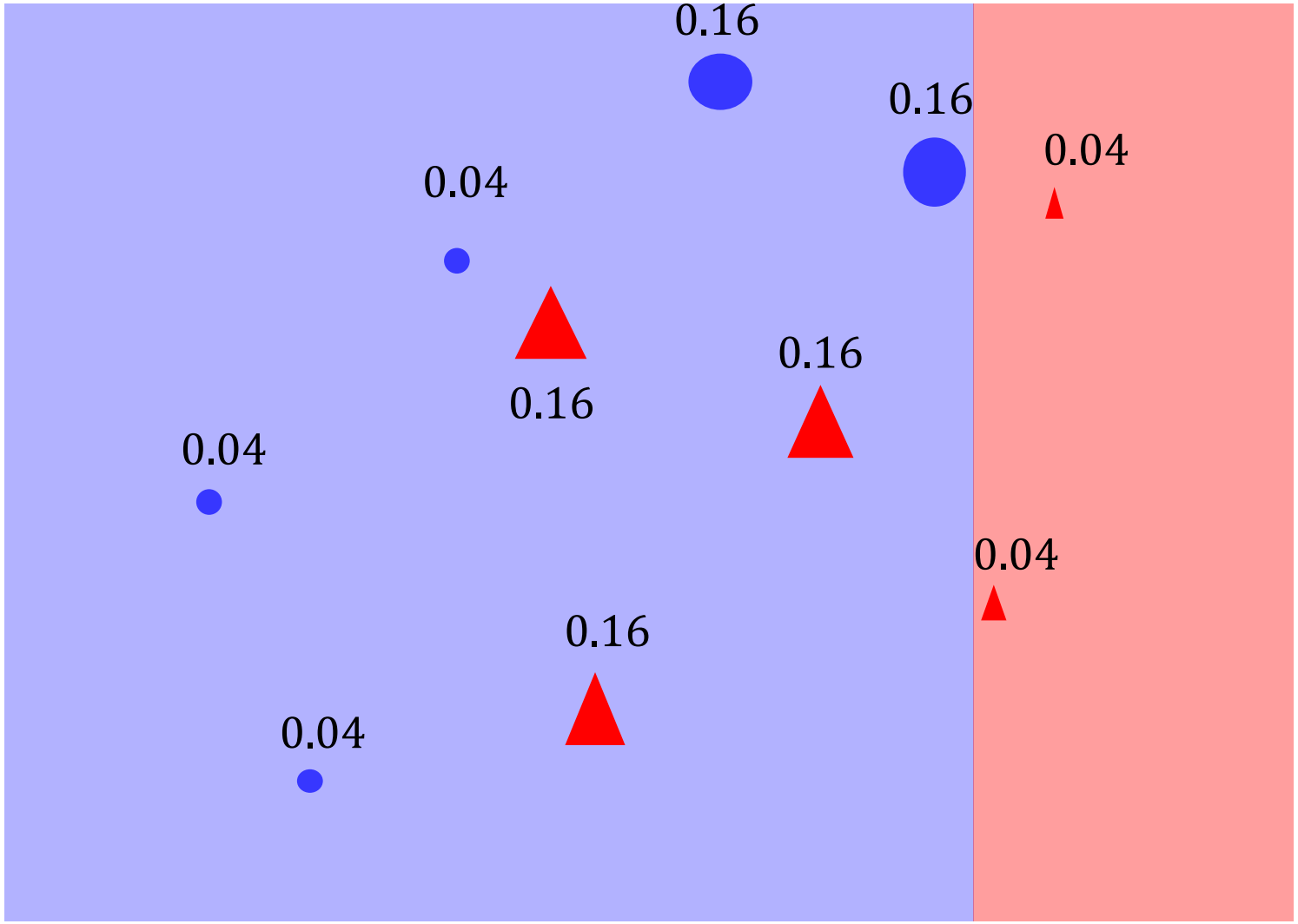
$e^{\alpha_2} \approx 2.08$

$e^{-\alpha_2} \approx 0.48$

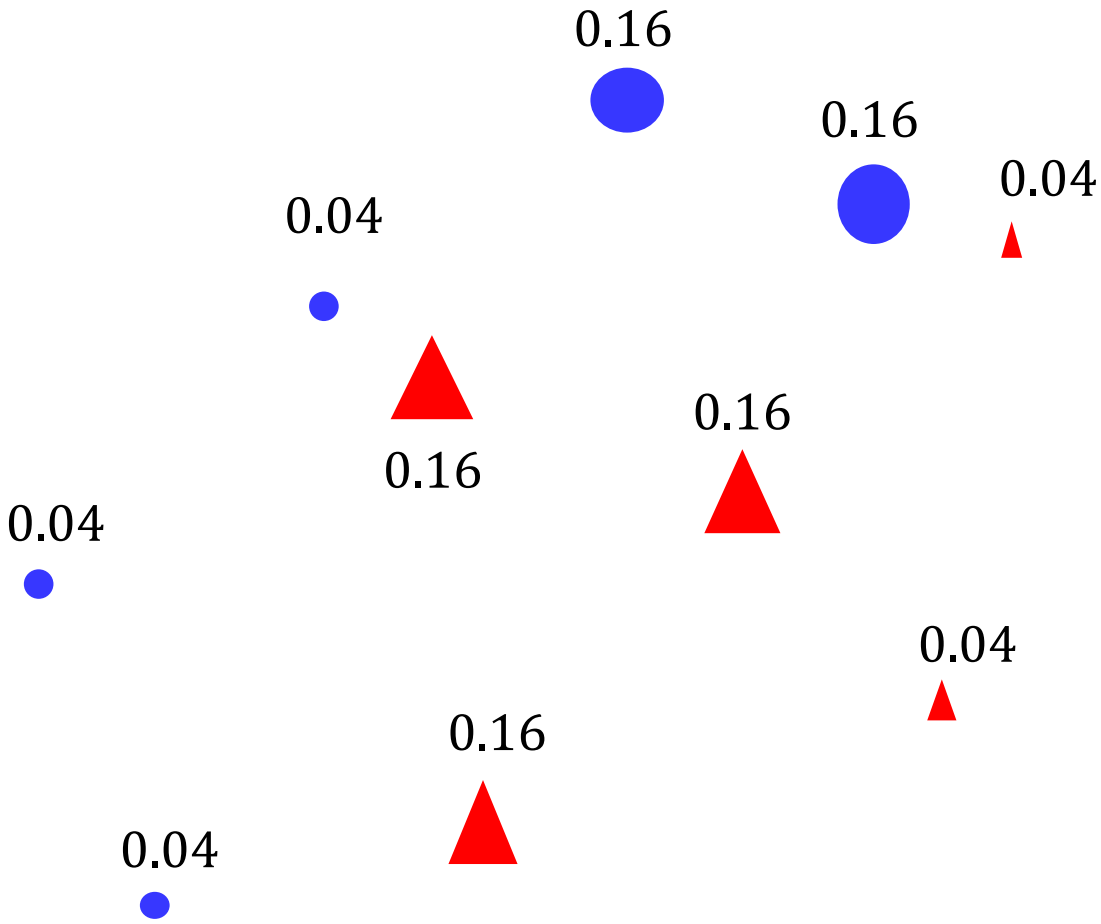


$Z_2 \approx 2 \cdot 0.25 \cdot 0.48 + 3 \cdot 0.06 \cdot 2.08 + 5 \cdot 0.06 \cdot 0.48 \approx 0.76$

AdaBoost with Decision Stumps



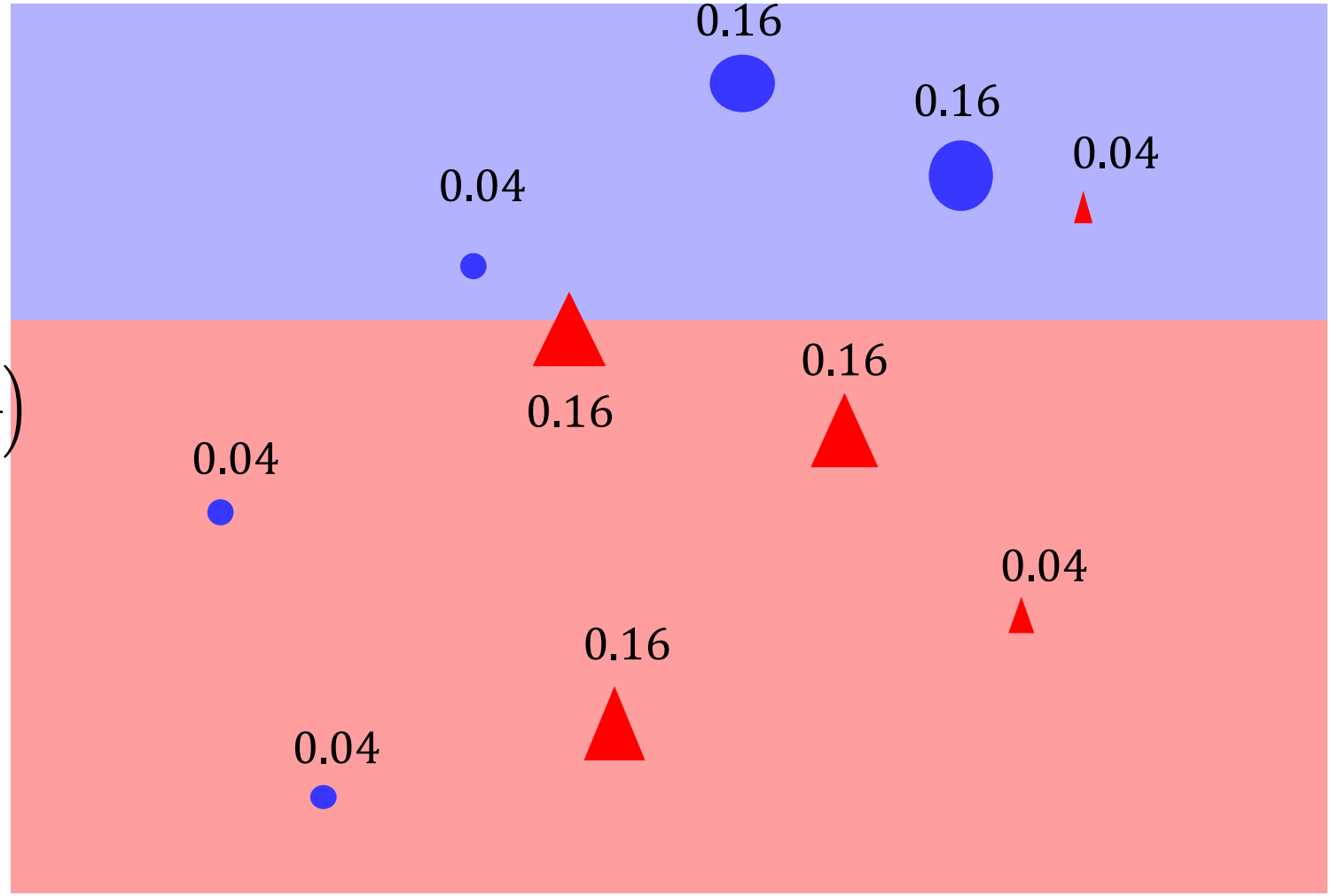
AdaBoost with Decision Stumps



AdaBoost with Decision Stumps

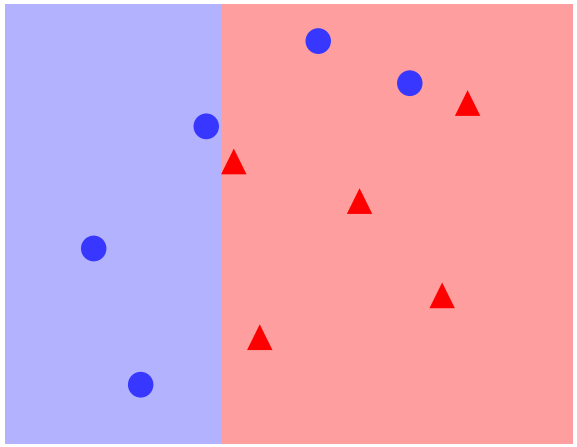
$er_3 \approx 0.12$

$\alpha_3 = \frac{1}{2}\ln\left(\frac{1-er_3}{er_3}\right) \approx 1.00$

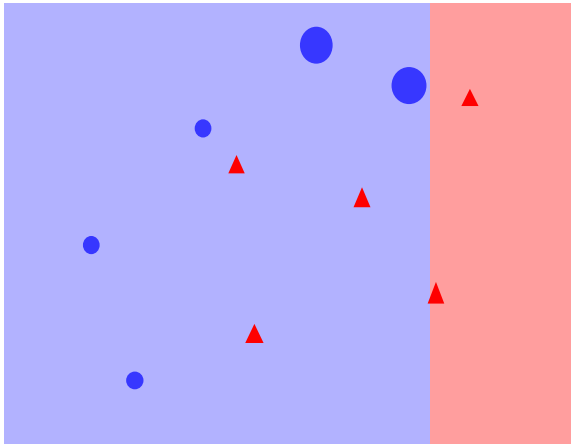


Let's stop after T=3 iterations

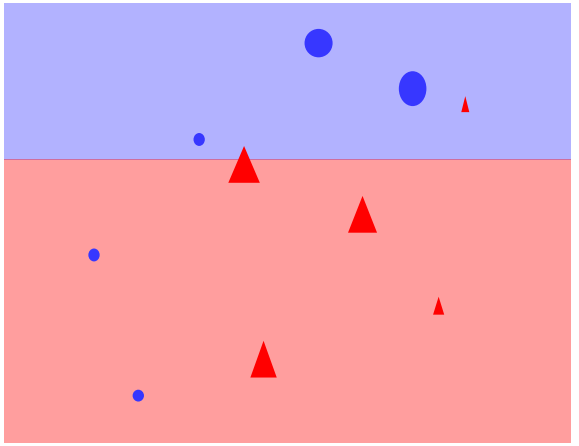
AdaBoost with Decision Stumps



$\alpha_1 \approx 0.69$

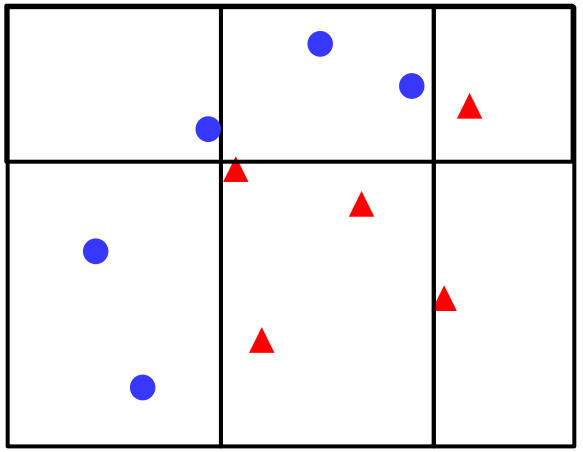


$\alpha_2 \approx 0.73$

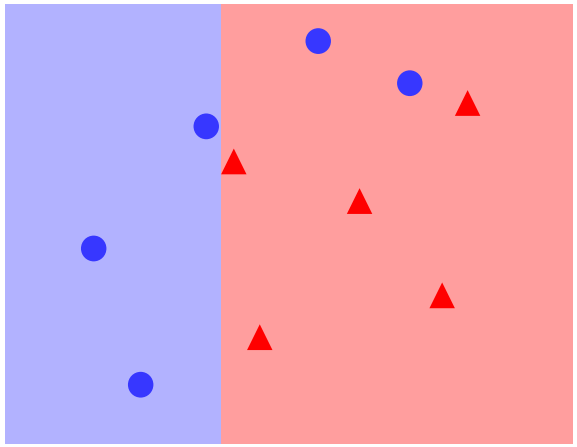


$\alpha_3 \approx 1.00$

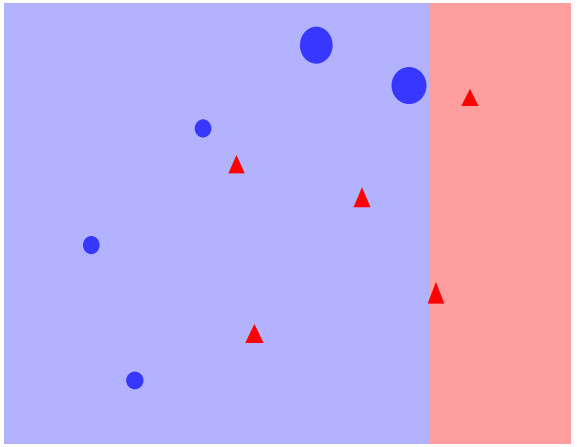
$$f(x) = \text{sign}(0.69h_1(x) + 0.73h_2(x) + 1.00h_3(x))$$



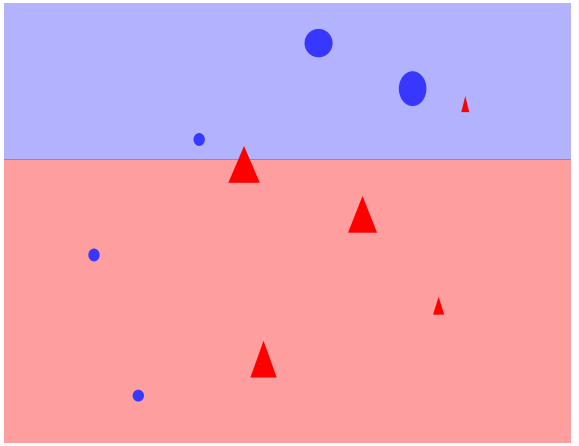
AdaBoost with Decision Stumps



$\alpha_1 \approx 0.69$



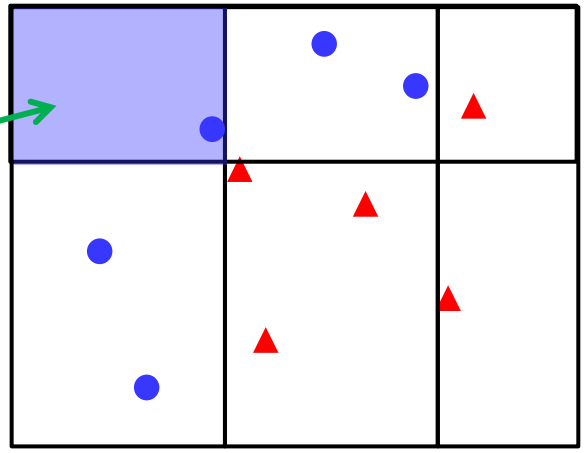
$\alpha_2 \approx 0.73$



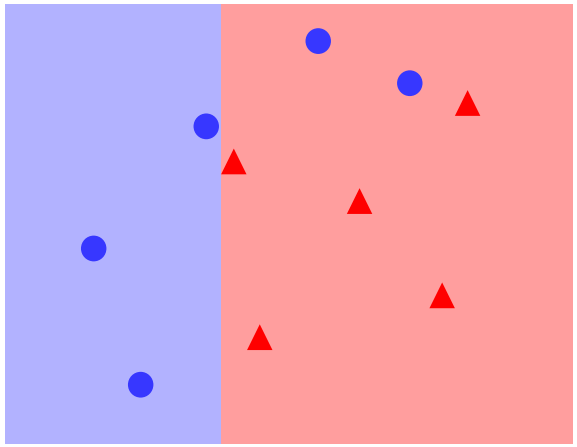
$\alpha_3 \approx 1.00$

$$f(x) = \text{sign}(0.69h_1(x) + 0.73h_2(x) + 1.00h_3(x))$$

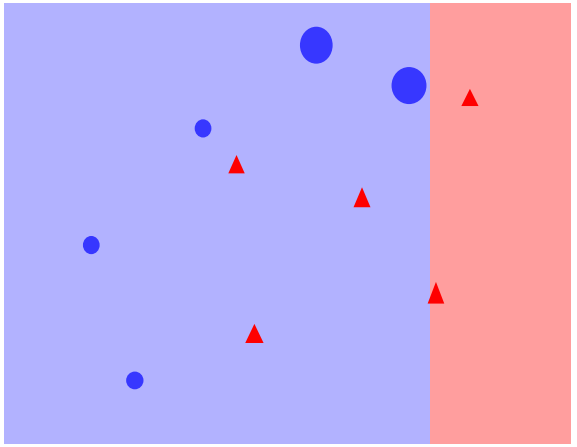
$\text{sign}(0.69 + 0.73 + 1.00) = 1$



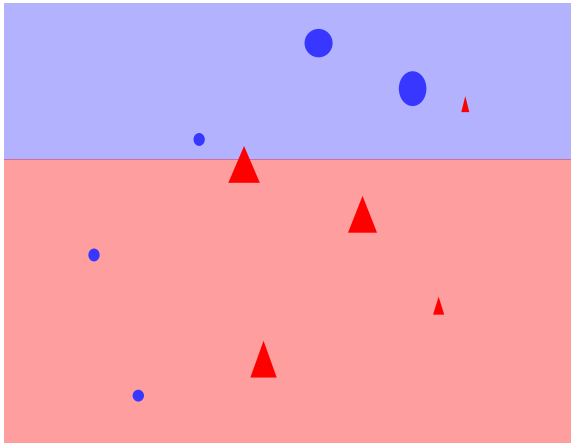
AdaBoost with Decision Stumps



$\alpha_1 \approx 0.69$

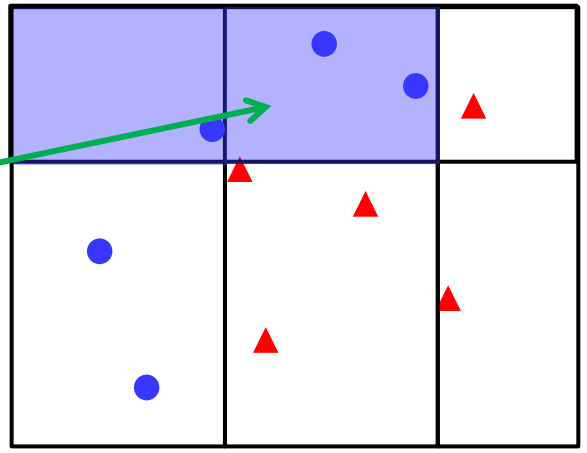


$\alpha_2 \approx 0.73$



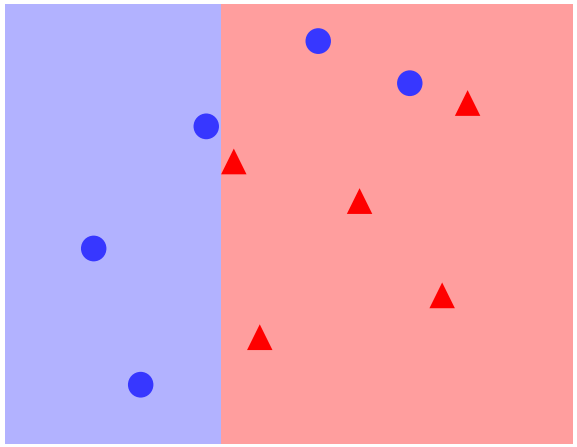
$\alpha_3 \approx 1.00$

$$f(x) = \text{sign}(0.69h_1(x) + 0.73h_2(x) + 1.00h_3(x))$$

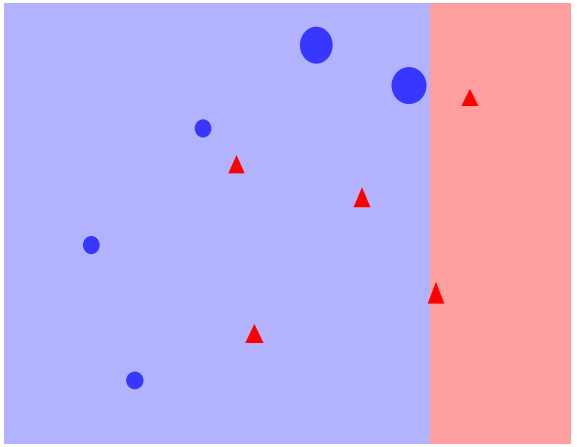


$\text{sign}(-0.69 + 0.73 + 1.00) = 1$

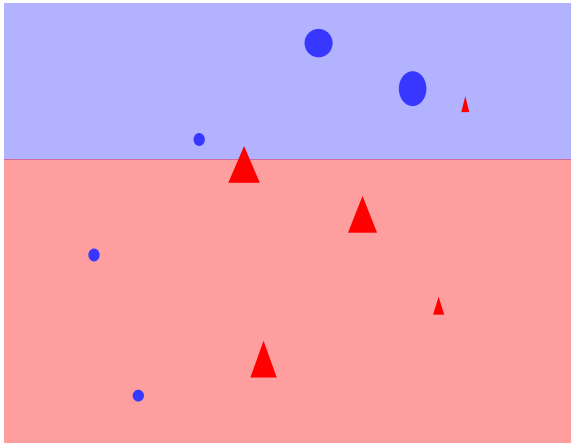
AdaBoost with Decision Stumps



$\alpha_1 \approx 0.69$

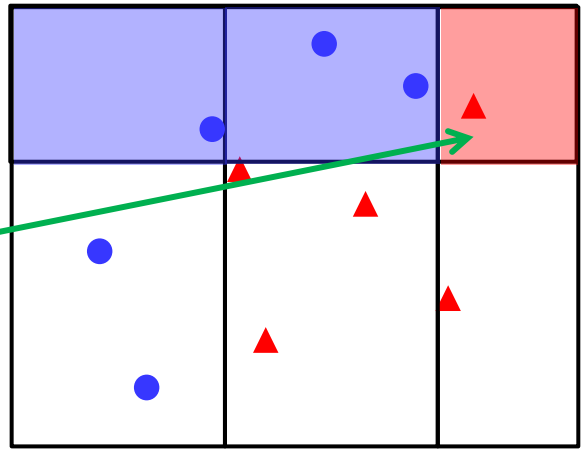


$\alpha_2 \approx 0.73$



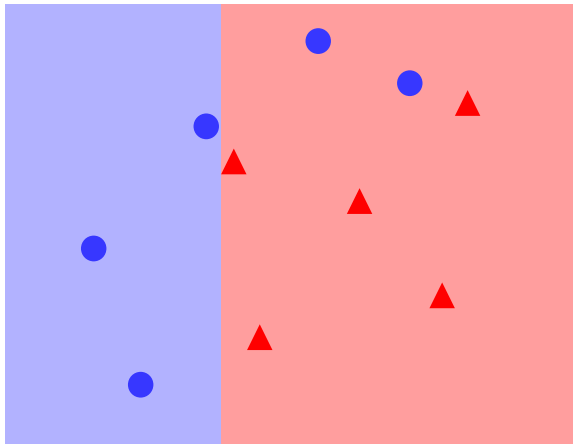
$\alpha_3 \approx 1.00$

$$f(x) = \text{sign}(0.69h_1(x) + 0.73h_2(x) + 1.00h_3(x))$$

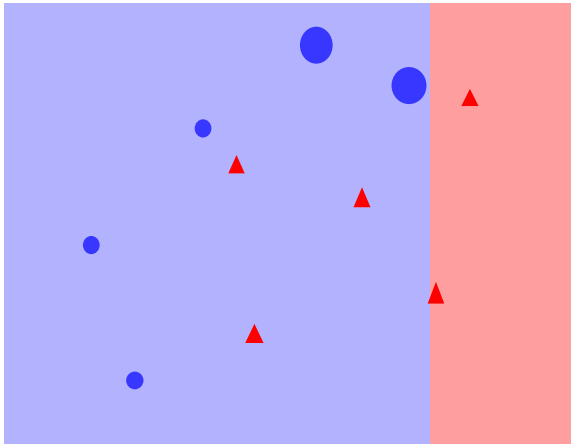


$\text{sign}(-0.69 - 0.73 + 1.00) = -1$

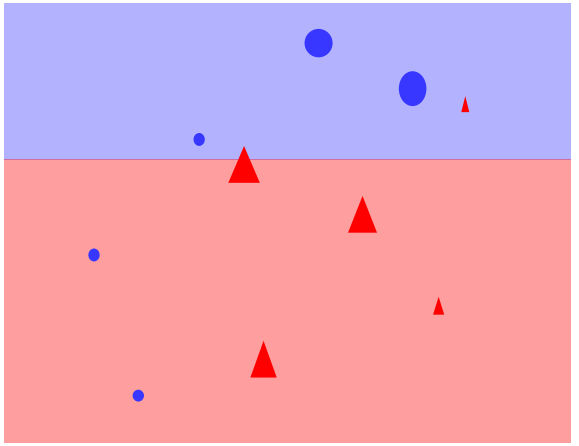
AdaBoost with Decision Stumps



$\alpha_1 \approx 0.69$



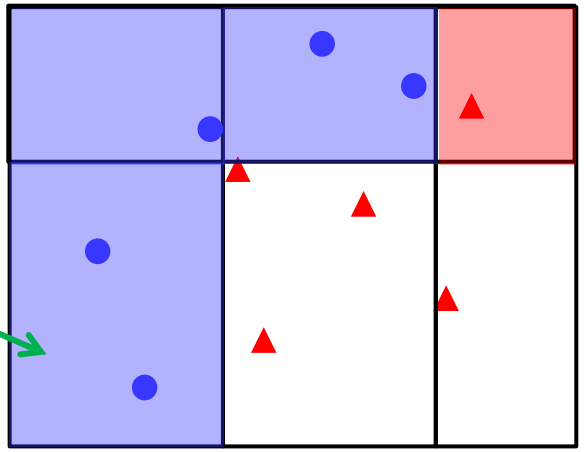
$\alpha_2 \approx 0.73$



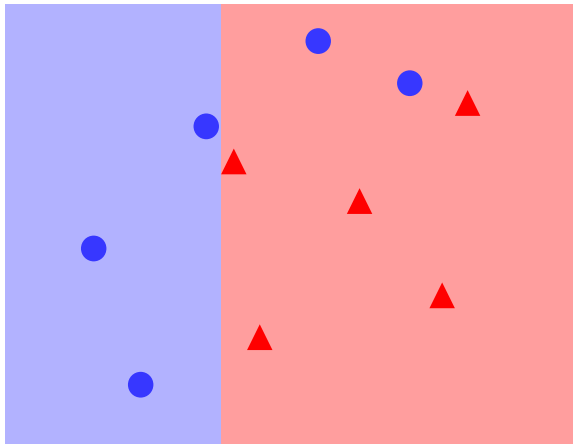
$\alpha_3 \approx 1.00$

$$f(x) = \text{sign}(0.69h_1(x) + 0.73h_2(x) + 1.00h_3(x))$$

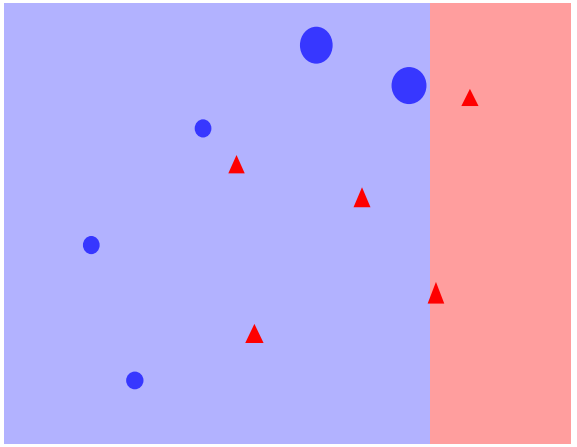
$\text{sign}(0.69 + 0.73 - 1.00) = 1$



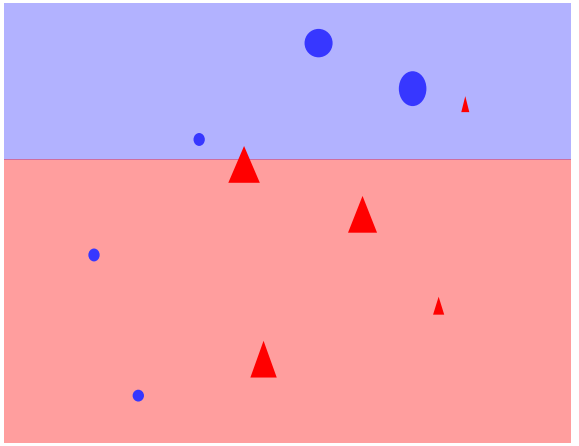
AdaBoost with Decision Stumps



$\alpha_1 \approx 0.69$



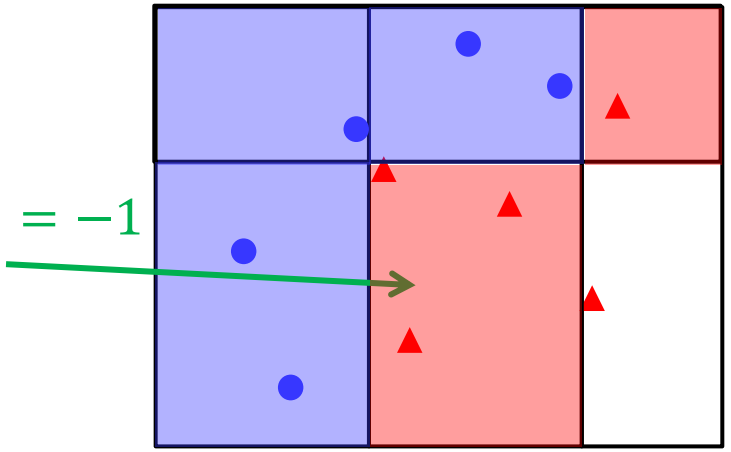
$\alpha_2 \approx 0.73$



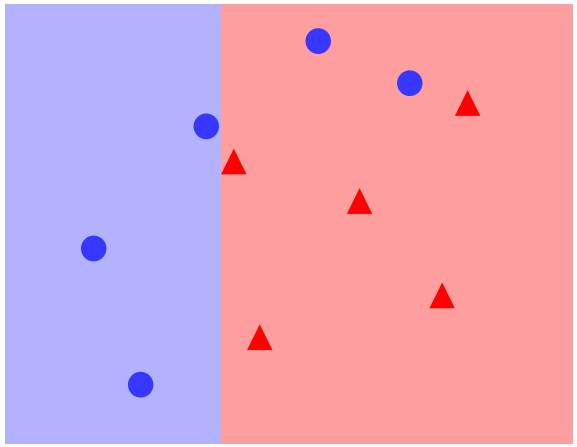
$\alpha_3 \approx 1.00$

$$f(x) = \text{sign}(0.69h_1(x) + 0.73h_2(x) + 1.00h_3(x))$$

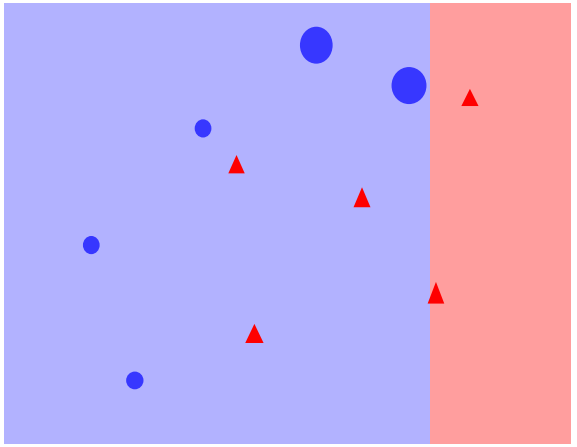
$\text{sign}(-0.69 + 0.73 - 1.00) = -1$



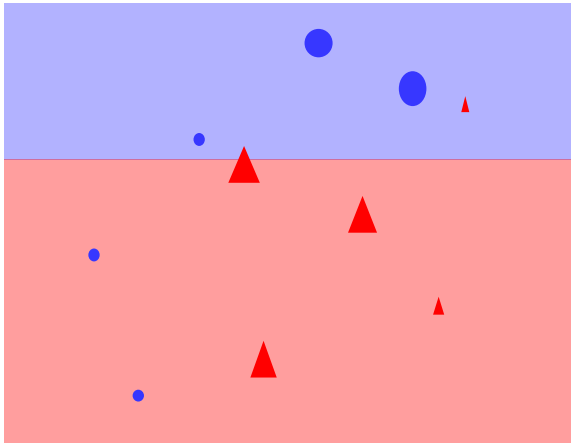
AdaBoost with Decision Stumps



$\alpha_1 \approx 0.69$

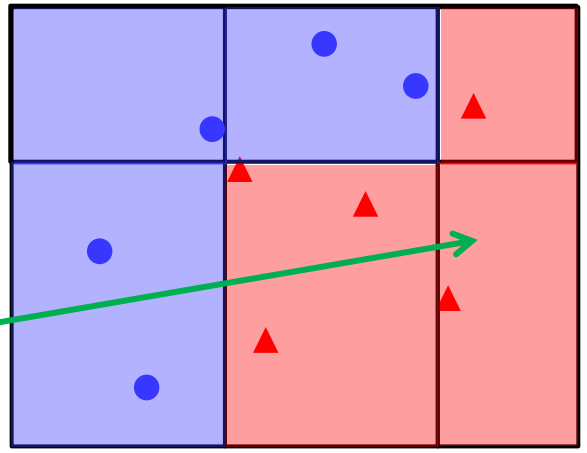


$\alpha_2 \approx 0.73$



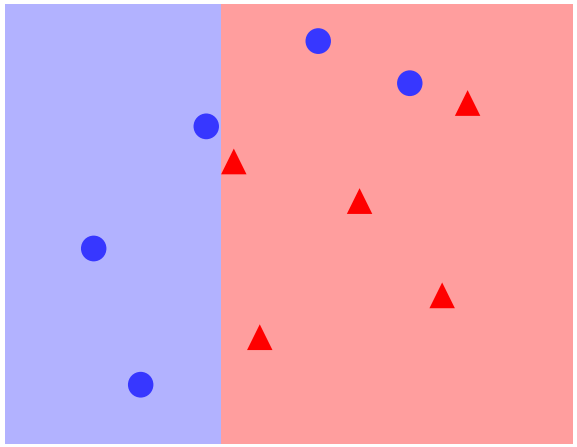
$\alpha_3 \approx 1.00$

$$f(x) = \text{sign}(0.69h_1(x) + 0.73h_2(x) + 1.00h_3(x))$$

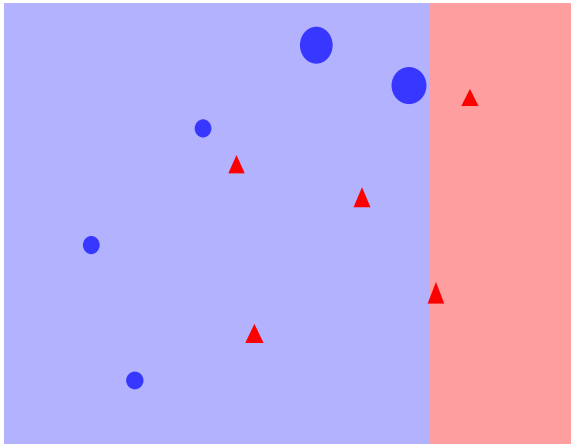


$\text{sign}(-0.69 - 0.73 - 1.00) = -1$

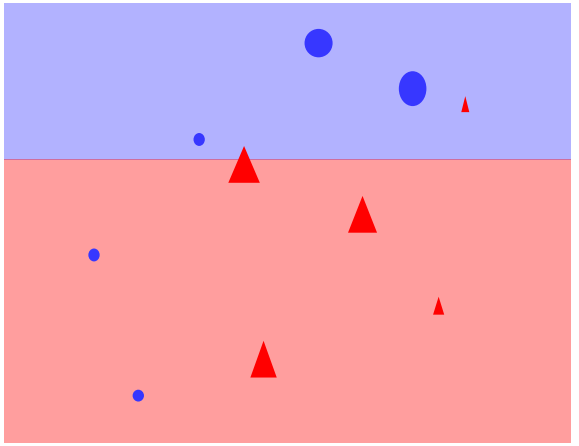
AdaBoost with Decision Stumps



$\alpha_1 \approx 0.69$

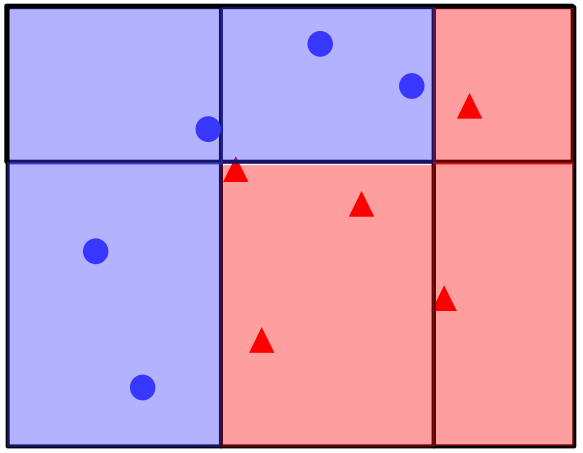


$\alpha_2 \approx 0.73$



$\alpha_3 \approx 1.00$

$$f(x) = \text{sign}(0.69h_1(x) + 0.73h_2(x) + 1.00h_3(x))$$



AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.
- Output: $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



AdaBoost

Lecture Note on Boosting

AARHUS
UNIVERSITY

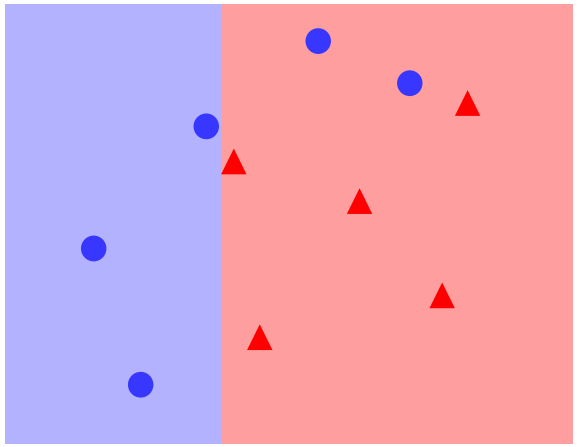
AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

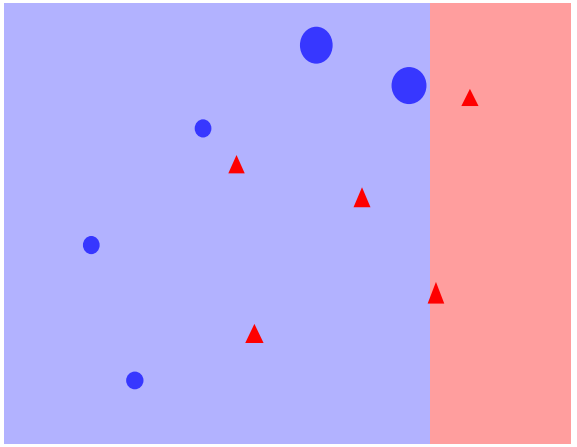
      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.
- Output: $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



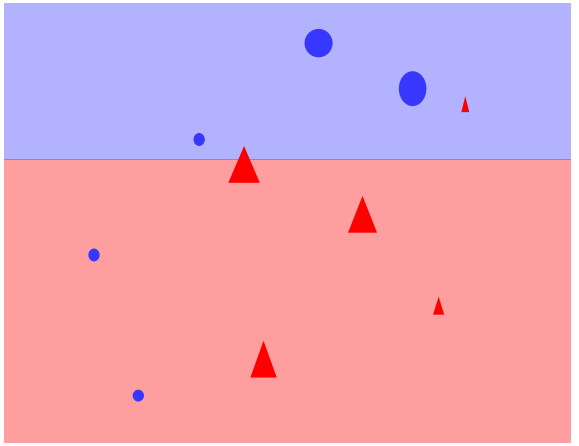
AdaBoost with Decision Stumps



$\alpha_1 \approx 0.69$

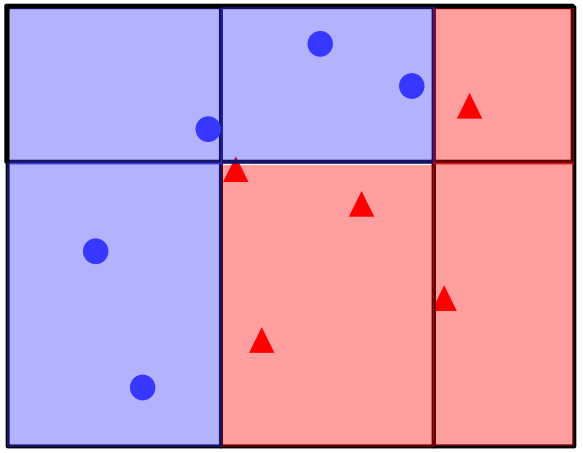


$\alpha_2 \approx 0.73$

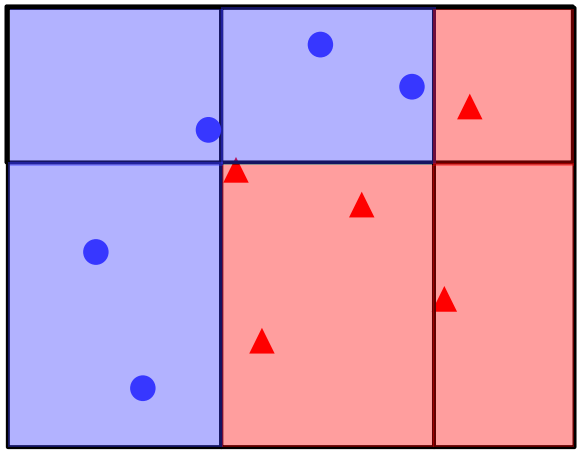


$\alpha_3 \approx 1.00$

$$f(x) = \text{sign}(0.69h_1(x) + 0.73h_2(x) + 1.00h_3(x))$$

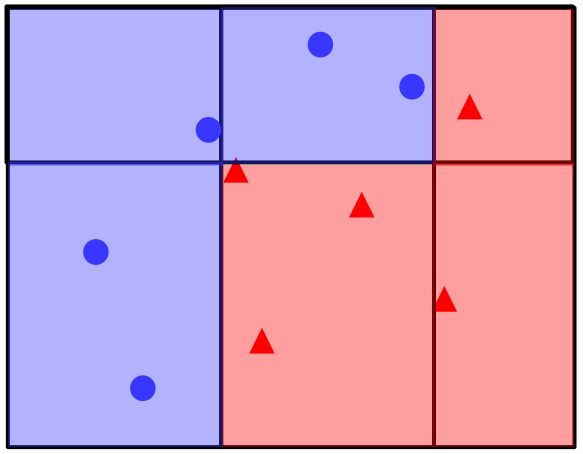


AdaBoost



- Weak learner is a classifier that does slightly better than guessing on training set.
- Strong learner is a classifier that perfectly classifies training set.
- AdaBoost:
  - Make weak learner into strong learner.
- Let us now prove it!

AdaBoost



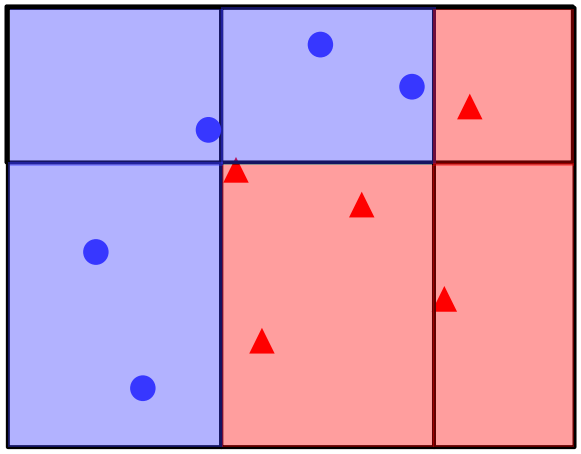
- Weak learner is a classifier that, for any weighing D of the training data, produces a hypothesis h that has $L_D(h) = \sum_{i=1}^{n} D(i) 1_{h(x_i) \neq y_i} < \frac{1}{2} - \gamma$ for some $\gamma > 0$.

- Strong learner is a classifier that perfectly classifies training set.

- AdaBoost:
  - Given weak learner base algorithm A, produces voting classifier f that is a strong learner.
- Let us now prove it!

If run for enough iterations T.

AdaBoost



- Notation:
  - We let

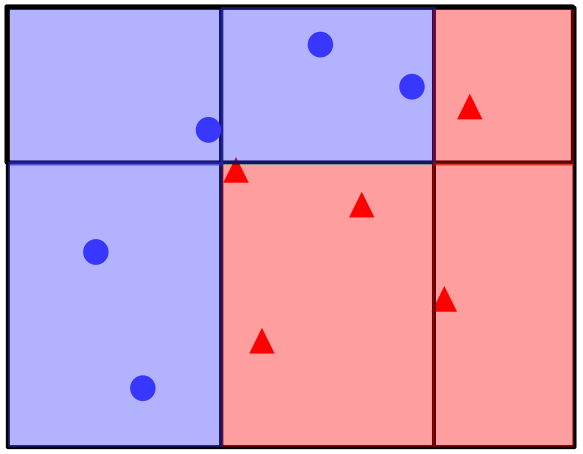
$$F_t(x) = \sum_{j=1}^{t} \alpha_j h_j(x).$$

    be such that the AdaBoost classifier after t rounds is $\text{sign}(F_t(x))$.

- Final Classifier:
  - $f(x) = \text{sign}(F_T(x))$

- Goal: Prove that for T large enough:
  - $E_{in}(f) = \frac{1}{n}\sum_{i=1}^{n} 1_{f(x_i)\neq y_i} = 0$

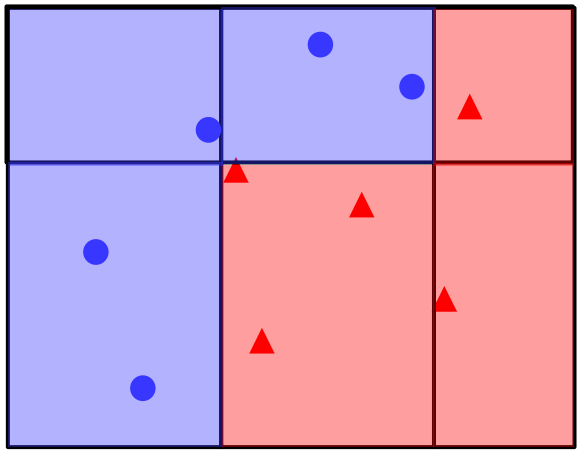
The Weighings



- Examine weighings:
  - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$
- Let us expand it:
  - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$
  - $= \frac{D_{t-1}(i)\exp(-\alpha_{t-1} y_i h_{t-1}(x_i))\exp(-\alpha_t y_i h_t(x_i))}{Z_{t-1}Z_t}$
  - $= \frac{D_1(i)\prod_{j=1}^{t}\exp(-\alpha_j y_i h_j(x_i))}{\prod_{j=1}^{t} Z_j}$
  - $= \frac{\frac{1}{n}\exp(\sum_{j=1}^{t} -\alpha_j y_i h_j(x_i))}{\prod_{j=1}^{t} Z_j}$
  - $= \frac{\frac{1}{n}\exp(-y_i F_t(x_i))}{\prod_{j=1}^{t} Z_j}$

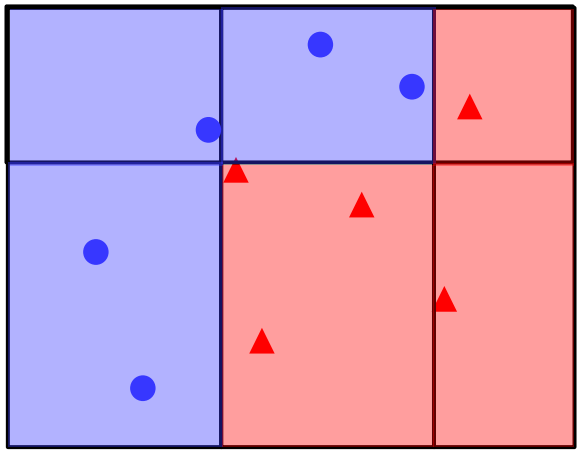


The Weighings



- Examine weighings:
  - $D_{t+1}(i) = \frac{\frac{1}{n}\exp(-y_i F_t(x_i))}{\prod_{j=1}^{t} z_j}$
- Observations:
  - If y_i and $F_t(x_i)$ have same sign, then weight is small.
    - $-y_i F_t(x_i) = -|F_t(x_i)|$
  - If y_i and $F_t(x_i)$ have opposite sign, then weight is large.
    - $-y_i F_t(x_i) = |F_t(x_i)|$
  - Like claimed earlier, AdaBoost focusses on points not classified well so far.

Bounding E_{in}

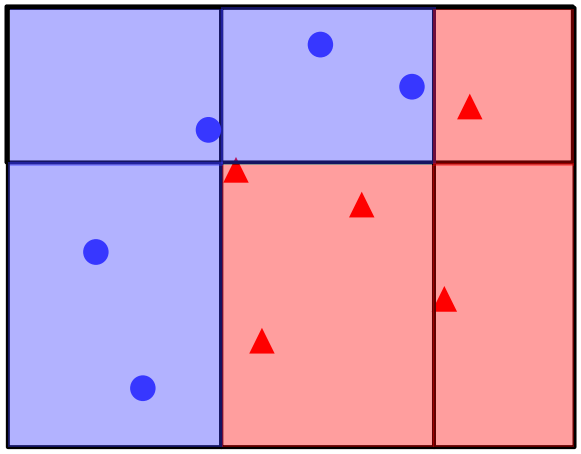


- Goal: Prove that for T large enough:
  - $\mathrm{E}_{\mathrm{in}}(f) = \frac{1}{n}\sum_{i=1}^{n} 1_{f(x_i)\neq y_i} = 0$
- Weighings satisfy:
  - $D_{t+1}(i) = \frac{\frac{1}{n}\exp(-y_i F_t(x_i))}{\prod_{j=1}^{t} Z_j}$
- Observation:
  - $1_{f(x_i)\neq y_i} =$
  - $1_{y_i F_T(x_i)\leq 0} \leq$
  - $\exp(-y_i F_T(x_i))$

If $y_i F_T(x_i)>0$, then indicator is 0 and exp()>0

If $y_i F_T(x_i)\leq 0$, then indicator is 1 and exp() is taken on non-negative number, so at least 1.

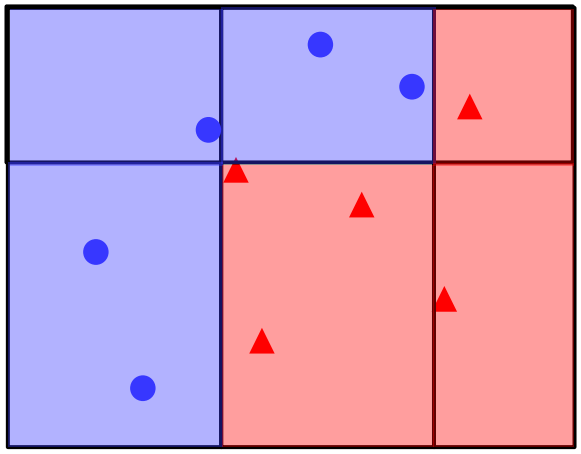
Bounding E_{in}



- Have:
  - $\mathrm{E}_{\mathrm{in}}(f) = \frac{1}{n}\sum_{i=1}^{n} 1_{f(x_i)\neq y_i} \leq \frac{1}{n}\sum_{i=1}^{n} \exp(-y_i F_T(x_i))$
- Weighings satisfy:
  - $D_{t+1}(i) = \frac{\frac{1}{n}\exp(-y_i F_t(x_i))}{\prod_{j=1}^{t} Z_j}$
- D_{T+1} is a probability distribution so:
  - $1 = \sum_{i=1}^{n} D_{T+1}(i) = \sum_{i=1}^{n} \frac{\frac{1}{n}\exp(-y_i F_T(x_i))}{\prod_{j=1}^{T} Z_j} \Rightarrow$
  - $\prod_{j=1}^{T} Z_j = \sum_{i=1}^{n} \frac{1}{n}\exp(-y_i F_T(x_i))$



Bounding E_{in}

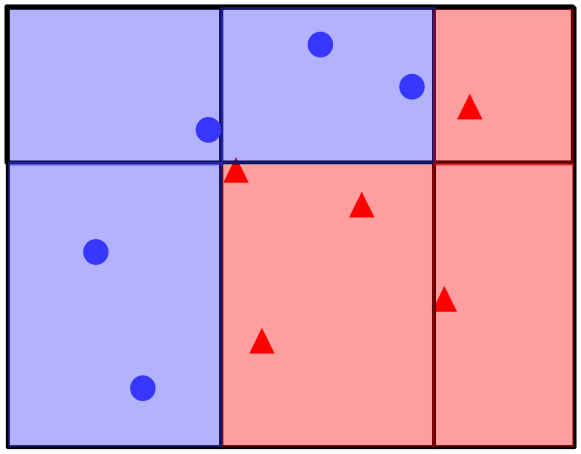


- Have:
  - $\mathrm{E}_{\mathrm{in}}(f) = \frac{1}{n}\sum_{i=1}^{n} 1_{f(x_i)\neq y_i} \leq \frac{1}{n}\sum_{i=1}^{n} \exp(-y_i F_T(x_i)) \leq \prod_{j=1}^{T} Z_j$
- Weighings satisfy:
  - $D_{t+1}(i) = \frac{\frac{1}{n}\exp(-y_i F_t(x_i))}{\prod_{j=1}^{t} Z_j}$
- D_{T+1} is a probability distribution so:
  - $1 = \sum_{i=1}^{n} D_{T+1}(i) = \sum_{i=1}^{n} \frac{\frac{1}{n}\exp(-y_i F_T(x_i))}{\prod_{j=1}^{T} Z_j} \Rightarrow$
  - $\prod_{j=1}^{T} Z_j = \sum_{i=1}^{n} \frac{1}{n}\exp(-y_i F_T(x_i))$



Bounding E_{in}

- Have:
  - $\mathrm{E}_{\mathrm{in}}(f) \leq \prod_{j=1}^{T} Z_j$
- Almost done:
  - We only need to bound $\prod_{j=1}^{T} Z_j$



AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

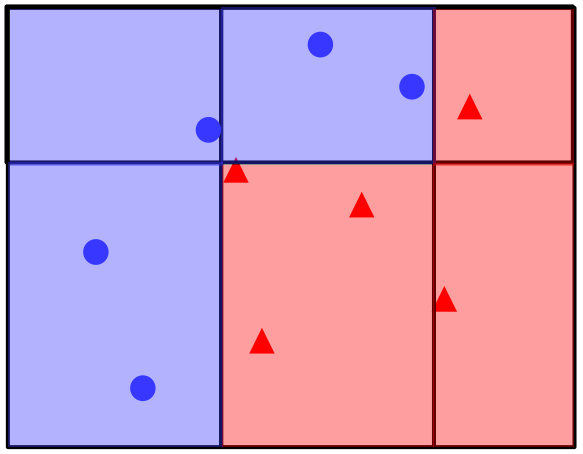
      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.
- Output: $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



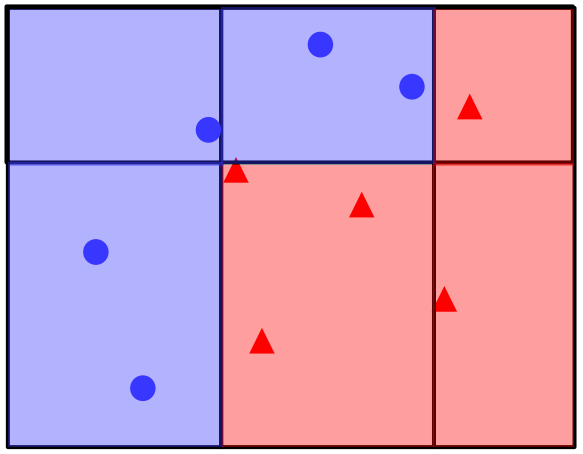
Bounding E_{in}

- Have:
  - $\mathrm{E}_{\mathrm{in}}(f) \leq \prod_{j=1}^{T} Z_j$

- Almost done:
  - We only need to bound $\prod_{j=1}^{T} Z_j$
  - $\mathrm{er_t} = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i) \neq y_i}$
  - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$
  - $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$



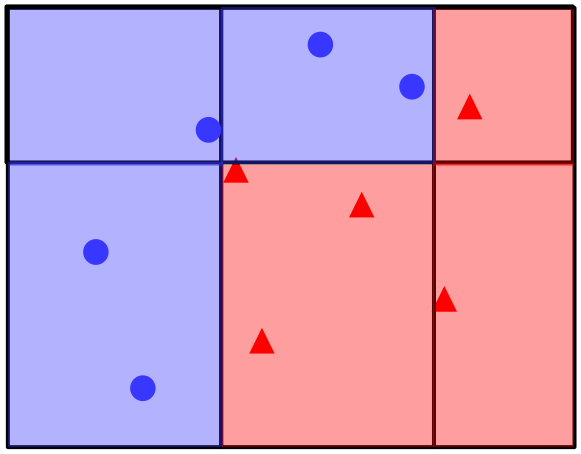
Bounding E_{in}



- Bounding Z_t:
  - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i) \neq y_i}$
  - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$
  - $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$
  - $= \sum_{j=1}^{n} D_t(j)(1_{y_j = h_t(x_j)}\, e^{-\alpha_t} + 1_{y_j \neq h_t(x_j)} e^{\alpha_t})$
  - $= e^{-\alpha_t} \sum_{j=1}^{n} D_t(j)\, 1_{y_j = h_t(x_j)} + e^{\alpha_t} \sum_{j=1}^{n} D_t\,(j) 1_{y_j \neq h_t(x_j)}$
  - $= e^{-\alpha_t}(1 - er_t) + e^{\alpha_t} er_t$
  - $= \sqrt{\frac{er_t}{1-er_t}}(1 - er_t) + \sqrt{\frac{1-er_t}{er_t}}\, er_t$
  - $= 2\sqrt{er_t(1 - er_t)}$



Bounding E_{in}



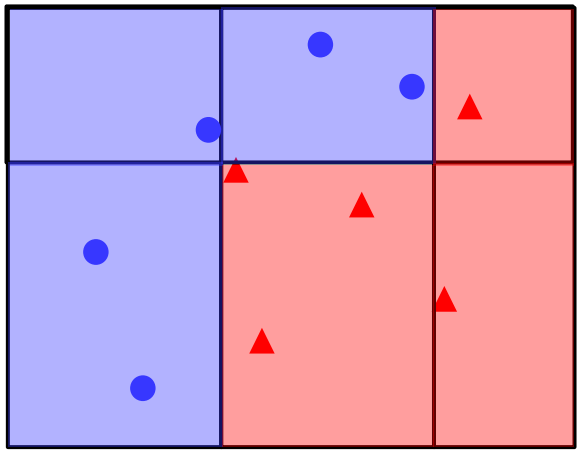
- Have:
  - $\mathrm{E}_{\mathrm{in}}(f) \leq \prod_{j=1}^{T} Z_j \leq \prod_{j=1}^{T} 2\sqrt{er_j(1-er_j)}$
- Weak Learner!
  - We assumed weak learner, i.e.
    - $er_t \leq \frac{1}{2} - \gamma$ for all t.
- Claim: $2\sqrt{er_t(1-er_t)}$ is increasing in er_t for $er_t<1/2$.
- To see this:
  - Derivative is, by chain rule:
    - $\frac{1}{\sqrt{er_t(1-er_t)}}(-2er_t + 1)$

All terms positive for $er_t<1/2$

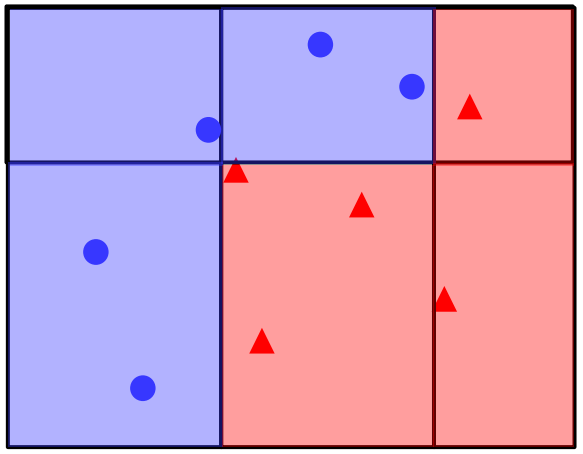
Bounding E_{in}

- Have:
  - $\mathrm{E}_{\mathrm{in}}(f) \leq \prod_{j=1}^{T} Z_j \leq \prod_{j=1}^{T} 2\sqrt{er_j(1-er_j)}$
  - $\leq \left(2\sqrt{\left(\frac{1}{2}-\gamma\right)\left(\frac{1}{2}+\gamma\right)}\right)^T$
  - $= \left(\sqrt{1-4\gamma^2}\right)^T$
  - $= (1-4\gamma^2)^{T/2}$

- Use that $1-x \leq e^{-x}$ for all x:
  - $\mathrm{E}_{\mathrm{in}}(f) \leq e^{-2T\gamma^2}$



Bounding E_{in}

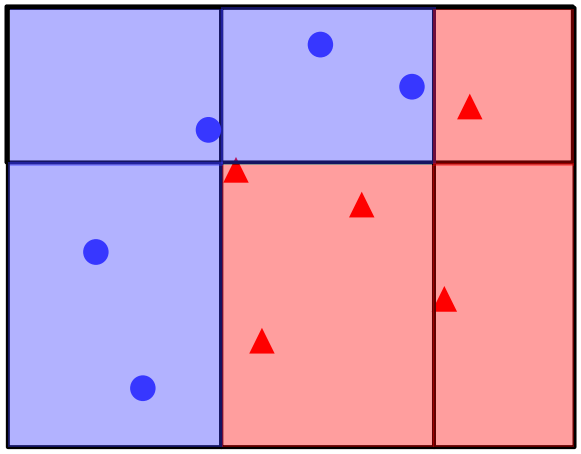


- Have:
  - $\mathrm{E}_{\mathrm{in}}(f) \leq e^{-2T\gamma^2}$
- Recall:
  - $\mathrm{E}_{\mathrm{in}}(f) = \frac{1}{n}\sum_{i=1}^{n} 1_{f(x_i)\neq y_i}$
- Thus:
  - If $\mathrm{E}_{\mathrm{in}}(f) < \frac{1}{n}$ then must have $\mathrm{E}_{\mathrm{in}}(f) = 0$.
- $e^{-2T\gamma^2} < \frac{1}{n}$
- $\Leftrightarrow -2T\gamma^2 < \ln(\frac{1}{n})$
- $\Leftrightarrow 2T\gamma^2 > \ln(n)$
- $\Leftrightarrow T > \frac{\ln(n)}{2\gamma^2}$



AdaBoost

- Have Shown:
  - If $er_t \leq \frac{1}{2} - \gamma$ for all t then AdaBoost has $E_{in}=0$ after $\frac{\ln(n)}{2\gamma^2} + 1$ iterations.



- Conclusion:
  - Turns weak learner into strong learner if run for enough iterations.

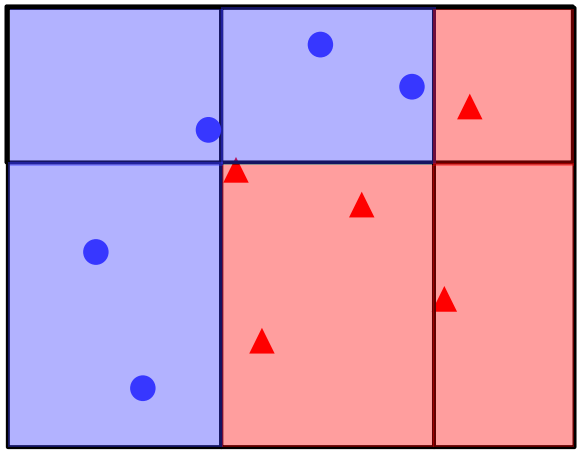
AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$. "magic" constant?
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.
- Output: $f(x) = \mathrm{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



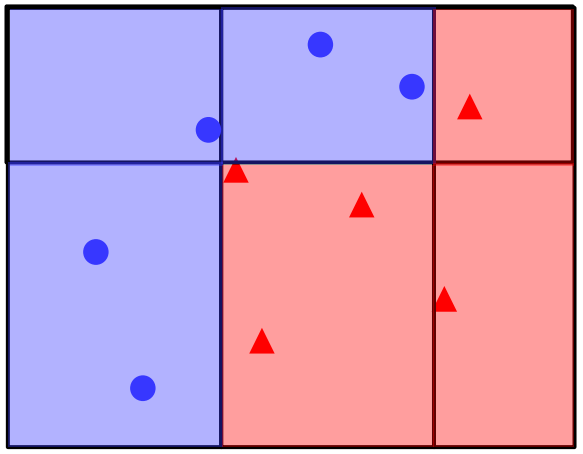
Choosing α_t



- Recall:
  - $\mathrm{E}_{\mathrm{in}}(f) \leq \prod_{j=1}^{T} Z_j$
- Bounding Z_t:
  - $\mathrm{er_t} = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i) \neq y_i}$
  - $Z_t = \sum_{j=1}^{n} D_t(j) \exp(-\alpha_t y_j h_t(x_j))$
  - $= \sum_{j=1}^{n} D_t(j)(1_{y_j = h_t(x_j)}\, e^{-\alpha_t} + 1_{y_j \neq h_t(x_j)} e^{\alpha_t})$
  - $= e^{-\alpha_t} \sum_{j=1}^{n} D_t(j)\, 1_{y_j = h_t(x_j)} + e^{\alpha_t} \sum_{j=1}^{n} D_t\,(j) 1_{y_j \neq h_t(x_j)}$
  - $= e^{-\alpha_t}(1 - \mathrm{er_t}) + \mathrm{e}^{\alpha_t}\mathrm{er_t}$
- Choose α_t to minimize this!



Choosing α_t



- Choose α_t to minimize:
  - $g(\alpha_t) = e^{-\alpha_t}(1 - \mathrm{er_t}) + \mathrm{e}^{\alpha_t}\mathrm{er_t}$
- Step 1: Convex function of α_t
  - $g'(\alpha_t) = -e^{-\alpha_t}(1 - \mathrm{er_t}) + e^{\alpha_t}\mathrm{er_t}$
  - $g''(\alpha_t) = e^{-\alpha_t}(1 - \mathrm{er_t}) + e^{\alpha_t}\mathrm{er_t}$
  - $g''(\alpha_t) > 0$ for all α_t so convex.
- Step 2: Minimize by setting $g'(\alpha_t) = 0$
  - $-e^{-\alpha_t}(1 - \mathrm{er_t}) + e^{\alpha_t}\mathrm{er_t} = 0 \Leftrightarrow$
  - $e^{-\alpha_t}(1 - \mathrm{er_t}) = e^{\alpha_t}\mathrm{er_t} \Leftrightarrow$
  - $e^{2\alpha_t} = (1 - \mathrm{er_t})/\mathrm{er_t} \Leftrightarrow$
  - $\alpha_t = \frac{1}{2}\ln((1 - \mathrm{er_t})/\mathrm{er_t})$



AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.
- Output: $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$

